HARDWARE AND SYSTEMS

Hardware and low-level systems from first principles

UNIT 2

Logic, Processors and Firmware

From transistors to a CPU, firmware and a measured boot chain

2.1 Digital logic

2.2 HDL and FPGA

2.3 Build a CPU

2.4 Memory and interconnect

2.5 Privilege and the platform

2.6 Bare-metal firmware

2.7 Boot and platform firmware

Second edition, October 2026

Contents

About this unit 2
Before you start 2
Kit for this unit 2
2.1 Digital logic 3
Core ideas 3
Reading 5
Labs 6
Problem set 8
Deliverables and stretch 8
2.2 HDL and FPGA 9
Core ideas 9
Reading 11
Labs 11
Problem set 13
Deliverables and stretch 13
2.3 Build a CPU 15
Core ideas 15
Reading 17
Labs 17
Problem set 19
Deliverables and stretch 19
2.4 Memory and interconnect 21
Core ideas 21
Reading 24
Labs 25
Problem set 26
Deliverables and stretch 27
2.5 Privilege and the platform 28
Core ideas 28
Reading 30
Labs 30
Problem set 32
Deliverables and stretch 32
2.6 Bare-metal firmware 33
Core ideas 33
Reading 36
Labs 36
Problem set 39
Deliverables and stretch 39
2.7 Boot and platform firmware 41
Core ideas 41
Reading 44
Labs 45
Problem set 47
Deliverables and stretch 47
Sign-off 49

UNIT 2

About this unit

Unit 2 builds a computer from the transistor up and then boots one you did not build, knowing every step. You make gates from MOSFETs, a pipelined RISC-V CPU on an FPGA with caches and privilege, and bare-metal firmware for TI's MSP432E401Y with a signed A/B bootloader. On the server you build a boot chain from a boot sector to Secure Boot, measured boot, remote attestation, netboot and OpenBMC. Board A in Unit 3 runs this unit's firmware, and Module Zero in Unit 4 joins its fleet through this unit's attestation.

Chapter	You build	Weeks
2.1	CMOS gates from transistors, an adder in logic ICs, a setup-time violation, a state machine	2
2.2	A UART, an asynchronous FIFO with a formal proof, a memory-mapped peripheral over SPI	3
2.3	A pipelined RV32IM CPU passing riscv-tests and running your C on the FPGA	4
2.4	Caches on your CPU, an SDRAM controller, memory experiments, PCIe by hand	3
2.5	M-mode and U-mode on your CPU, x86-64 bare metal, ACPI and SMBIOS read by hand	2
2.6	Library-free MSP432E401Y firmware with DMA, fault handling and a signed A/B bootloader	3.5
2.7	UEFI loader, Secure Boot, measured boot and attestation, netboot, Redfish and OpenBMC	3.5

Before you start

Finish the Handbook's setup checklist for the FPGA tools, OpenOCD and QEMU. Order the server during Unit 1 so it is set up before Chapter 2.4.

Kit for this unit

The Handbook's kit part lists every item for this unit with its part number, seller and price. The totals, for items this unit needs first:

	Required	Optional
UK sellers, VAT included (ex-VAT prices × 1.2)	£420	£0
Overseas sellers, before import VAT and delivery	US$275	none
Items with no checked price	1	none

Not priced: Used rack server: Dell PowerEdge R730 (or R630, or Supermicro X11).

CHAPTER 2.1

Digital logic

TIME	2 weeks
YOU BUILD	A CMOS inverter and NAND from discrete MOSFETs; a 4-bit adder in SN74HC logic with its ripple delay measured; a captured setup-time violation; a sequence detector designed with Karnaugh maps and built in gates; a small datapath in the Digital simulator.
NEEDS	Bench kit, logic analyser, 2N7000 and BS250 MOSFETs, the SN74HC logic set (00, 04, 08, 14, 32, 74, 86), DIP switches and LEDs, the Digital simulator.

WHAT THIS BUYS OMH

Every SERDES, memory controller, BMC and glue FPGA in an OMH module is built from the objects in this chapter: gates, flip-flops and the timing constraints between them. You will hear "timing closure", "metastability", "clock domain crossing" and "setup violation" from FPGA engineers and silicon vendors. By the end of this chapter you will have caused each of them on a breadboard and fixed it.

SKIP TEST

1. Draw a CMOS NAND gate at the transistor level and explain why its output is driven strongly in both states.
2. Given $t_{cq} = 1$ ns, $t_{logic} = 6$ ns, $t_{setup} = 0.5$ ns and $t_{hold} = 0.3$ ns, what is the maximum clock frequency? When does a hold violation occur, and does slowing the clock fix it?
3. Why does a two-flop synchroniser reduce the metastability failure rate but not to zero? Write the MTBF expression.
4. Design a Moore machine that detects the sequence 1-0-1 on a serial input. How many states? Is one-hot or binary encoding better on an FPGA, and why?
5. Why does a 6T SRAM cell need exactly six transistors, and what are the two that are not part of the inverter pair for?

If all five are easy, do Labs 2.1.1 and 2.1.3 only.

Core ideas

Bits are voltage ranges with margins. A 74HC input treats anything below V_{IL} as 0 and above V_{IH} as 1, and outputs guarantee V_{OL} and V_{OH}. The gap between an output's guarantee and an input's threshold is the noise margin, and the rail noise of Chapter 1.1 uses it up. Mixed 3.3 V and 5 V logic fails when V_{OH} of one family is below V_{IH} of the other, which is why level shifters exist.

CMOS: complementary pairs. An N-channel MOSFET pulls low when its gate is high, and a P-channel pulls high when its gate is low. Arranged in series and parallel they make an inverter, a NAND or a NOR. In steady state one network is off, so almost no current flows, and power is spent charging and discharging capacitance at each transition. Dynamic power is roughly $CV^2 f$, which is why lowering the voltage saves the most power and why a data-centre CPU's power scales with clock and activity.

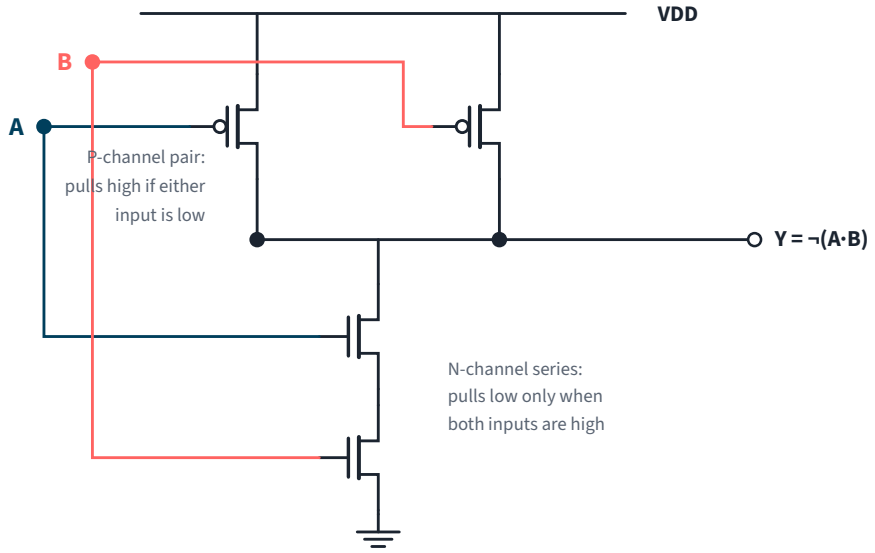


Figure 2.1.1. A CMOS NAND gate. The two P-channel transistors in parallel pull the output high if either input is low; the two N-channel transistors in series pull it low only when both are high. Lab 2.1.1 builds it from 2N7000 and BS250 parts.

NAND is universal. Any Boolean function can be built from NANDs alone, or from NORs. Synthesis tools pick from a library of gates, but the problem is tractable because one gate suffices.

Boolean algebra and minimisation. Learn sum of products, product of sums, De Morgan, Karnaugh maps and don't-cares. You will rarely minimise by hand after this chapter, but you must be able to read what a synthesiser did and judge whether it is what you meant.

Combinational building blocks. The standard blocks are the multiplexer, decoder, encoder, comparator, half and full adder, ripple-carry adder, carry-lookahead adder and ALU. Ripple-carry delay grows linearly with the width, and lookahead delay grows logarithmically at the cost of more area. Every datapath makes this trade.

Propagation delay and the critical path. Every gate has a propagation delay. The longest path from any input to any output is the critical path, and it bounds how fast the block can run. Synthesis timing reports are lists of critical paths.

Latches and flip-flops. Cross-coupled NANDs form an SR latch, the first bistable. Gate it with an enable and you have a D latch, transparent while enabled. Two latches back to back on opposite clock phases make an edge-triggered D flip-flop, which samples only at the clock edge. Sequential logic is combinational logic between flip-flops that share one clock.

Timing: setup, hold and the period constraint. Data must be stable t_{setup} before the edge and t_{hold} after it. The period must satisfy $T \geq t_{\text{cq}} + t_{\text{logic,max}} + t_{\text{setup}}$. Hold requires $t_{\text{cq}} + t_{\text{logic,min}} \geq t_{\text{hold}}$, and the period does not appear, so a hold violation cannot be fixed by slowing the clock. Clock skew between two flops takes margin from one constraint and gives it to the other.

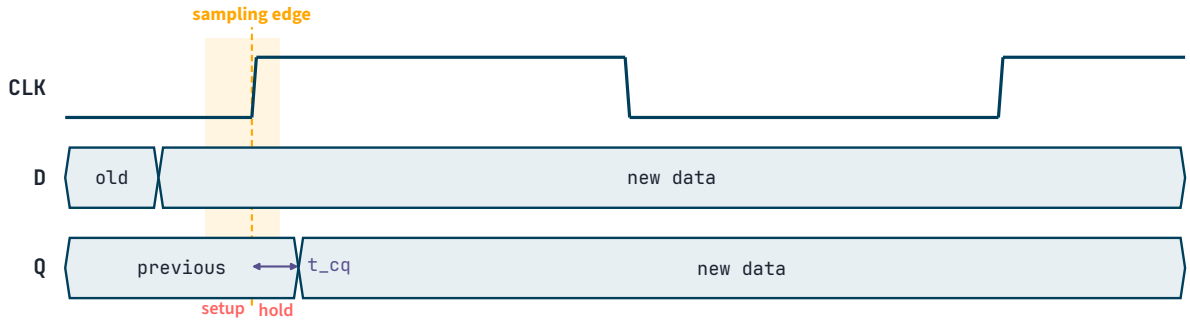


Figure 2.1.2. Setup and hold. Data launched by one clock edge must arrive t_{setup} before the next and stay t_{hold} after it. Lab 2.1.3 sweeps the data edge through this window and watches the flop fail.

Metastability. If a flop samples an input that is changing at the edge, its output can sit between 0 and 1 for an unbounded time before resolving. This cannot be prevented for asynchronous inputs, but a two-flop synchroniser makes failure rare, with $\text{MTBF} = e^{t_r/\tau}/(T_0 f_{\text{clk}} f_{\text{data}})$, where t_r is the time allowed to resolve. Every reset line, button and signal crossing between clock domains is asynchronous.

Finite state machines. A state machine has a state register, next-state logic and output logic. Moore outputs depend on the state only, and Mealy outputs also depend on the inputs. One-hot encoding uses more flops and simpler, faster logic, which is why it is preferred on FPGAs, where flops are plentiful.

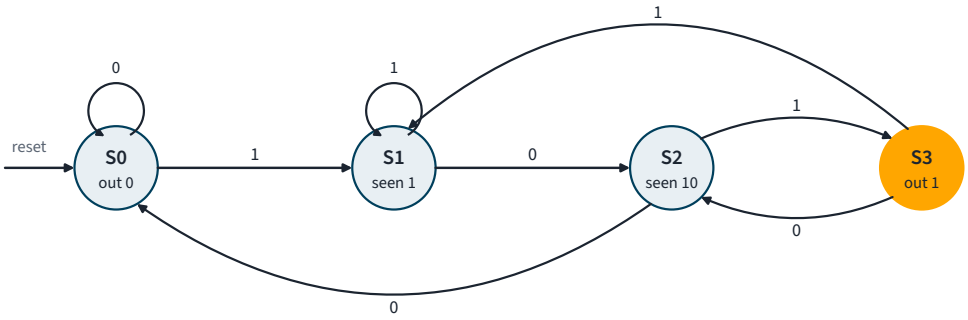


Figure 2.1.3. The Moore machine for Lab 2.1.4, detecting 1-0-1 with overlap. Each state is named for the longest useful suffix seen so far; only the final state asserts the output.

Registers, counters and shift registers. A register is flops sharing a clock. A counter is a register plus an incrementer. A shift register moves bits one position per clock, and UARTs, SPI and every other serial protocol use one to turn parallel data into serial and back.

Memory. A 6T SRAM cell is two cross-coupled inverters and two access transistors, selected by a word line and read or written on bit lines. A register file is a small SRAM with several ports. A ROM is a decoder driving fixed pull-downs. Tri-state outputs let many blocks drive one wire at different times.

Reset and asynchronous inputs. Synchronous reset is a data input. Asynchronous reset acts at once, but its release must be synchronised or flops leave reset on different cycles. Glitches on combinational outputs between edges are harmless as long as nothing samples them, which is why clocked designs only look at the edge.

Fan-out and loading. Each input is a capacitive load. Driving many inputs slows the edge, and buffers restore drive.

Reading

- Harris and Harris, *Digital Design and Computer Architecture*, RISC-V edition: chapter 1, chapter 2 (combinational), chapter 3 (sequential, especially 3.5 timing and 3.6 parallelism), chapter 5 sections 5.1 to 5.5.
- Nand2Tetris Part 1, projects 1 to 3. They take a weekend and are worth it for the repetition.
- Ben Eater's 8-bit breadboard computer videos on the clock module, registers and the ALU. Watch for the debugging technique.
- Cummings, "Clock Domain Crossing (CDC) Design and Verification Techniques Using SystemVerilog", SNUG 2008. Read the first half now and the rest in Chapter 2.2.
- TI's SN74HC00, SN74HC74 and SN74HC14 datasheets: switching characteristics and the timing-waveform figures.

Labs

LAB 2.1.1 Gates from transistors — 4 h

Goal. build an inverter and a NAND from discrete MOSFETs and measure them.

Kit. 2N7000, BS250, potentiometer, multimeter, oscilloscope.

1. Build a CMOS inverter from a 2N7000 and a BS250 on 5 V. Drive the input from a potentiometer and plot V_{out} against V_{in} in 0.25 V steps. Find the switching threshold and the region of gain.
2. Build a two-input NAND (two P-channel in parallel, two N-channel in series). Verify its truth table with the meter.
3. Chain three inverters into a ring oscillator. Measure its frequency and compute the propagation delay per stage. Add 100 pF to one node and measure again.
4. Measure supply current at rest and while oscillating, and relate it to $CV^2 f$.

DONE WHEN

- [ ] You have the transfer curve, the NAND truth table, and a measured delay per stage with an explanation of what limits it.
- [ ] The current measurement is reconciled with $CV^2 f$ to within a factor of two, with the capacitance estimated from the 100 pF experiment.

EVIDENCE FOR THE NOTEBOOK

- Transfer-curve plot; truth table; oscillator captures with and without the capacitor.

LAB 2.1.2 A 4-bit adder in SN74HC logic — 6 h

Goal. build a ripple-carry adder, measure its delay and design a faster one.

Kit. SN74HC86N, SN74HC08N, SN74HC32N, DIP switches, LEDs with resistors.

1. Draw a full adder from XOR, AND and OR. Build four of them from SN74HC86N, SN74HC08N and SN74HC32N, with DIP switches for inputs and LEDs for sum and carry-out.
2. Verify against a table of 20 random additions.
3. Drive carry-in with a square wave and set $A = 1111$ and $B = 0000$. Capture carry-in against carry-out and measure the ripple delay.
4. On paper, design 4-bit carry-lookahead logic. Count gate levels on the critical path against the ripple version.

DONE WHEN

- [ ] The adder passes all 20 additions.
- [ ] The ripple delay is measured and compared with four times the datasheet's per-gate delays.
- [ ] The lookahead design is written with a gate-level delay comparison.

EVIDENCE FOR THE NOTEBOOK

- Wiring photograph; the addition table; delay capture; lookahead design.

LAB 2.1.3 Flip-flops and a setup-time violation — 5 h

Goal. cause and measure a setup-time violation on a real flip-flop.

Kit. SN74HC00N, SN74HC74N, SN74HC14N, a potentiometer and capacitor for the adjustable delay, oscilloscope or logic analyser.

1. Build an SR latch from SN74HC00N gates, then a gated D latch, then a master-slave D flip-flop from two D latches. Verify each with switches.
2. Clock an SN74HC74N as a divide-by-two and confirm it on the scope.

3. Feed the SN74HC74N's D input from a signal derived from the clock through an adjustable RC delay, so that its edge sweeps across the clock edge as you turn the pot. Watch Q and find the window in which it becomes unreliable. The width of that window is the setup time plus the hold time.
4. Put two flops in series. Show the second flop's output is clean while the first misbehaves.

DONE WHEN

- [ ] A capture shows the single flop failing, with the data-to-clock offset measured at the failure.
- [ ] The notebook explains why the two-flop version is better and why it is not perfect, using the MTBF expression.

EVIDENCE FOR THE NOTEBOOK

- Captures of the failure and of the two-flop result.

LAB 2.1.4 A finite state machine in gates — 5 h

Goal. design a state machine from a specification and build it in gates.

Kit. SN74HC74N, gates, SN74HC14N for debouncing, push button.

1. Specify a 1-0-1 sequence detector as a Moore machine. Draw the state diagram and transition table; choose a binary encoding.
2. Derive next-state and output equations with Karnaugh maps.
3. Build it from SN74HC74N flops and gates, clocked by hand through a debounced button (RC plus SN74HC14N). Verify with sequences that include overlaps.
4. Redo the design on paper with one-hot encoding and compare the logic.

DONE WHEN

- [ ] The breadboard machine detects 1-0-1 including overlapping occurrences.
- [ ] Both encodings are in the notebook with their equations and gate counts.

EVIDENCE FOR THE NOTEBOOK

- State diagram, K-maps, both sets of equations, a video or capture of a test sequence.

LAB 2.1.5 A datapath in Digital — 6 h

Goal. build the register file, ALU and datapath that Chapter 2.3 needs.

Kit. The Digital simulator by hneemann.

1. Build an 8-bit register file with two read ports and one write port, an 8-bit ALU (add, subtract, and, or, xor, shift), and a multiplexer-based datapath connecting them.
2. Add a small ROM as instruction memory with a hand-assembled encoding of your design: opcode, destination and two sources.
3. Add a program counter and a decoder. Run a six-instruction program that shows a result on an LED array. Single-step it.

DONE WHEN

- [ ] Your hand-encoded program runs and you can say what happens in each clock cycle.

EVIDENCE FOR THE NOTEBOOK

- The Digital file committed; a one-page description of your instruction encoding.

Problem set

1. Prove that NAND is functionally complete by constructing NOT, AND, OR and XOR from NANDs alone. Count gates for XOR.
2. A path has $t_{cq} = 0.8$ ns, combinational delay between 2.1 ns and 7.4 ns, $t_{setup} = 0.6$ ns and $t_{hold} = 0.4$ ns, and the receiving flop's clock arrives 0.5 ns after the sender's. Compute the maximum frequency and check hold. Repeat with the skew negated.
3. A synchroniser's flops have $\tau = 30$ ps and $T_0 = 20$ ps; the clock is 200 MHz; the asynchronous input toggles at 10 MHz. Compute MTBF for one flop and for two in series.
4. Design a 3-bit Gray-code counter and argue why Gray code matters for the asynchronous FIFO pointers in Chapter 2.2.
5. A CMOS block switches 50 pF per cycle at 1.0 V and 3 GHz. Compute dynamic power, then at 0.8 V and 2.4 GHz. Why do server CPUs have dozens of voltage and frequency states?
6. Build a 4:1 multiplexer from a 2-to-4 decoder and AND-OR logic, then from three 2:1 multiplexers. Compare gate counts and delay.
7. From the SN74HC04 datasheet at 3.3 V, extract V_{IH}, V_{IL}, V_{OH} and V_{OL} and compute both noise margins. Repeat for an SN74HC output driving a 5 V SN74HCT input, and for an SN74HCT output driving a 3.3 V SN74LVC input; say which pairing is safe.
8. Show that a ripple-carry adder's delay grows linearly with width and a carry-lookahead adder's logarithmically. Where does the area go?
9. A single-cycle "data valid" pulse must cross from a 100 MHz domain to a 33 MHz domain. Why does a synchroniser alone lose pulses, and what handshake fixes it?

Deliverables and stretch

Deliverables. The notebook with the transfer curve, the ripple delay, the setup-violation capture and the FSM derivation; the Digital datapath file, which Chapter 2.3 replaces with Verilog.

Stretch. Build a 4-bit SRAM from SN74HC573 latches with address decoding and SN74HC245 tri-state outputs, written and read from switches. Read the LVDS and CML sections of a SERDES primer and write half a page on why gigabit signalling abandoned single-ended CMOS levels.

COMPLETION CHECKLIST

- ☐ **Lab 2.1.1:** transfer curve, NAND truth table, delay per stage, and the current reconciled with CV^2f.
- ☐ **Lab 2.1.2:** adder verified on 20 cases; ripple delay measured against the datasheet; lookahead design written.
- ☐ **Lab 2.1.3:** a captured setup or hold failure with the offset measured; the two-flop result.
- ☐ **Lab 2.1.4:** the detector works with overlaps; both encodings derived.
- ☐ **Lab 2.1.5:** the Digital datapath runs a hand-assembled program.
- ☐ **Problem set:** all nine answered; problems 2 and 3 checked numerically.
- ☐ **You can explain**, without notes: why a hold violation is independent of the clock period, what metastability is and why synchronisers only reduce it, and why FPGAs prefer one-hot state machines.

CHAPTER 2.2

HDL and FPGA

TIME	3 weeks
YOU BUILD	A hand-constrained blinky with a deliberate timing failure; a UART with a formal proof; an asynchronous FIFO proved with induction; a memory-mapped peripheral with a written register map; that register map bridged to SPI and driven from the Raspberry Pi.
NEEDS	ULX3S FPGA board, OSS CAD Suite (Yosys, nextpnr, Verilator, SymbiYosys), logic analyser, Raspberry Pi 5.

WHAT THIS BUYS OMH

An OMH module will contain programmable logic: drive-bay backplane control, LED and presence signalling, power sequencing glue, or a test rig for a CXL or NVMe interface. Every controller chip you buy is also a design like the ones you write here, and its datasheet describes registers, clock domains and FIFOs that only make sense once you have built them. The register-map peripheral from Lab 2.2.4 starts a thread that runs through Chapter 2.6 (a microcontroller drives it), Chapter 3.4 (a stretch respin puts an ECP5 on the management board) and Chapter 4.3 (a Linux driver exposes it).

SKIP TEST

1. What is the difference between `=` and `<=` inside `always_ff`, and what goes wrong if you mix them?
2. Why can a synthesiser infer a latch from an `always_comb` block, and how do you make that impossible?
3. Describe how an asynchronous FIFO's full and empty flags are computed safely across two clock domains.
4. A path fails timing by 1.2 ns at 100 MHz. List four ways to fix it in the order you would try them.
5. What is the difference between a multicycle-path constraint and a false path, and why does lying to the tool about either produce hardware that works on the bench and fails in the field?

If all five are easy, do Labs 2.2.3 and 2.2.5 only.

Core ideas

HDL describes hardware. Every statement describes logic that exists and operates at the same time. A module is a circuit, and instantiating it twice makes two circuits. Coming from C++, the most important habit is to think of each line as the flops and gates it becomes, since nothing in it "executes" in sequence.

Two kinds of block. `always_ff @(posedge clk)` describes flip-flops; use non-blocking `<=` so that all right-hand sides are sampled before any left-hand side updates, which is how real flops behave. `always_comb` describes gates; use blocking `=` and assign every output on every path (or give a default) so that no latch is inferred. A signal is assigned in exactly one block.

Simulation and synthesis are different consumers. Verilator compiles your design into C++ for fast cycle-based simulation driven from a C++ testbench, which your background makes natural. Icarus is slower and event-driven and accepts more of the language. Yosys turns the same source into a netlist of LUTs and flops. Not everything simulable is synthesisable (`#` delays, unbounded loops).

FPGA architecture. Look-up tables of four to six inputs implement any small Boolean function, and each pairs with a flip-flop. Carry chains make adders fast. Block RAM gives dense memory with registered ports, DSP blocks multiply and accumulate, PLLs synthesise clocks, and I/O blocks handle voltage standards and serialisation. Programmable routing between them takes most of the die and most of the delay. A bitstream loaded from flash configures it all at power-on.

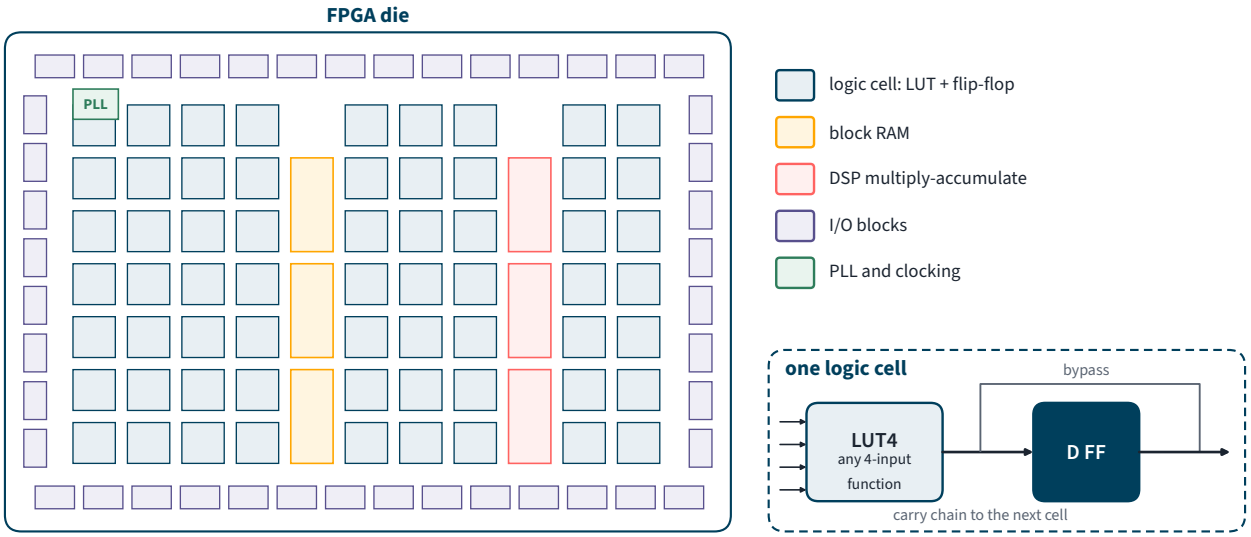


Figure 2.2.1. The parts of an FPGA fabric your designs map onto. Nextpnr's utilisation report counts each of these; its timing report measures the routing between them.

Timing analysis. You state the clock period in a constraints file. The tool computes every path from flop to flop and reports slack, required time minus arrival time. Negative slack is a failing path. Place-and-route is an optimisation problem, so results vary from run to run, and a design that only just passes may fail on the next build.

Clock domain crossing. A signal moving between unrelated clocks is asynchronous to the receiver. Single bits get a two-flop synchroniser. Multi-bit values must not be synchronised bit by bit, because the bits arrive on different cycles. Use a handshake, or a FIFO with Gray-coded pointers so that only one bit changes per increment. This is the most common cause of "works for hours, then corrupts" bugs.

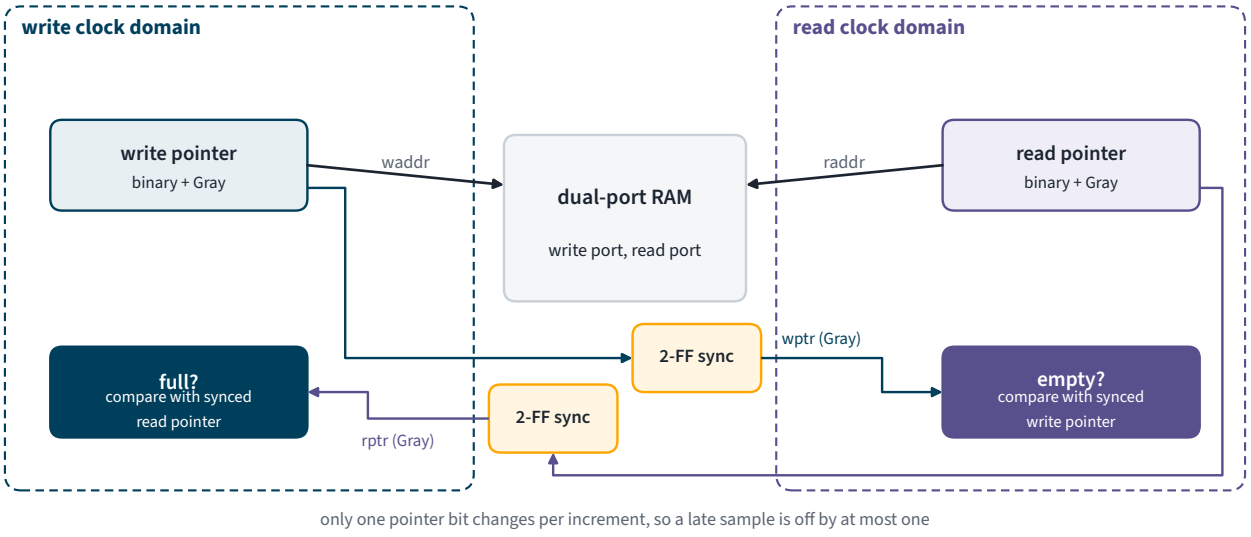


Figure 2.2.2. An asynchronous FIFO. Each pointer is Gray-coded in its own domain and crosses through a two-flop synchroniser; "full" is computed in the write domain and "empty" in the read domain, each against a possibly stale copy of the other pointer, which is always safe.

Reset. Synchronise reset release. Decide per design whether flops are reset at all. On FPGAs, initial values from the bitstream often remove the need, and unreset datapaths are smaller and faster.

Interfaces and buses. Wishbone (simple, open) and AXI-Lite (the industry standard, more signals) are memory-mapped register buses: address, data, strobe or valid, ready or acknowledge. AXI-Stream carries data without addresses. Every SoC peripheral you will read a datasheet for is one of these behind a register map. Write the register map before the RTL, because it is the contract with software.

Formal verification. SymbiYosys uses SAT solvers to prove properties of your design for all inputs up to a bound, or for ever with induction. For FIFOs, arbiters and protocol handlers it finds bugs that simulation never reaches.

Parameterisation and reuse. `parameter` and `generate` give you templates. Keep peripherals generic in width and depth, because Chapter 2.3 instantiates them.

Which language. SystemVerilog's synthesisable subset with Verilator is the industry default and this course's. VHDL is equally capable and common in defence and Europe. Amaranth (Python) and Chisel (Scala) generate Verilog, and LiteX builds whole SoCs from Python. Learn SystemVerilog first so you can read everyone else's code.

Vendor tools. The open flow covers only some parts, and the rest need Vivado (AMD), Quartus (Altera) or Radiant (Lattice). The concepts are the same, but the constraint syntax and the GUI differ.

Reading

- Harris and Harris, DDCA RISC-V edition, chapter 4 (HDL), the SystemVerilog columns.
- ZipCPU (Dan Gisselquist): "Building a simple UART", "Crossing clock domains", "Building a formally verified asynchronous FIFO" and the formal verification tutorial.
- Cummings, "Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!" and "Simulation and Synthesis Techniques for Asynchronous FIFO Design" (SNUG).
- Yosys manual chapters 1 to 3; the nextpnr README; the ULX3S constraints file and examples; the Verilator manual's C++ testbench sections.
- Wishbone B4 (short) and the AXI4-Lite chapter of the AMBA AXI specification.

Labs

LAB 2.2.1 Toolchain, blink and a timing failure on purpose — 5 h

1. Starting from scratch, with no example project, write a top module with a PLL from the 25 MHz input to 100 MHz, a 27-bit counter and the LEDs on its top bits. Write the LPF constraints by hand from the ULX3S schematic. Build with Yosys, nextpnr-ecp5 and ecppack, and load with openFPGALoader.
2. Read the nextpnr timing report end to end. Find the critical path and its slack.
3. Add a 64-bit combinational multiply feeding a flop. Read the negative slack. Fix it by pipelining; then fix it instead by lowering the PLL frequency, and note which fix you would choose and why.

DONE WHEN

- [ ] The LEDs count, and both timing reports are in the notebook with the critical paths annotated.

EVIDENCE FOR THE NOTEBOOK

- Constraints file; the two annotated reports.

LAB 2.2.2 UART transmitter and receiver — 8 h

1. Write the register map first: TX data, RX data, status (TX busy, RX valid, framing error) and baud divisor.

2. Implement 8N1 TX as a state machine with a shift register and a baud counter. Write a Verilator testbench in C++ that decodes the line and checks each byte.
3. Implement RX with 16× oversampling and mid-bit sampling. The testbench sends bytes with slightly wrong baud rates and finds the tolerance.
4. Prove with SymbiYosys that once a start bit is detected, exactly eight data bits are sampled and the stop bit is checked.
5. On hardware, echo at 115200 through the on-board FTDI. Capture the line with the logic analyser and decode it in PulseView.

DONE WHEN

- [ ] Echo works on the board, the formal proof passes, and the baud tolerance is measured in simulation.

EVIDENCE FOR THE NOTEBOOK

- RTL, testbench and proof files; the tolerance result; a decoded capture.

LAB 2.2.3 An asynchronous FIFO with a formal proof — 8 h

1. Implement a parameterised asynchronous FIFO: Gray-coded write and read pointers, two-flop synchronisers for the crossing pointers, full and empty computed in the correct domains.
2. Simulate with unrelated clocks (100 MHz and 37 MHz), random bursts and a scoreboard that checks order and count.
3. Prove with induction that the FIFO never overflows or underflows and delivers data in order.
4. Break the Gray coding (use binary pointers) and show the solver finds a counterexample.

DONE WHEN

- [ ] The proof passes with induction, and the broken version fails with a trace you can explain.

EVIDENCE FOR THE NOTEBOOK

- Proof files; the counterexample trace annotated.

LAB 2.2.4 A memory-mapped peripheral — 6 h

1. Define a Wishbone (or AXI-Lite) slave with this 32-bit register map: GPIO output, GPIO input, GPIO direction, timer counter, timer compare, timer control, interrupt status, interrupt enable, and an ID register that returns a constant.
2. Implement it. The testbench acts as bus master, walks the map, checks read-after-write, checks the timer counts and raises the interrupt at the compare value.
3. Write the register map as a table with bit fields, reset values and access types. It is the contract for Chapters 2.3, 2.6 and 4.3.

DONE WHEN

- [ ] The testbench passes and the register document is committed.

EVIDENCE FOR THE NOTEBOOK

- RTL, testbench, the register-map document.

LAB 2.2.5 The register map over SPI — 8 h

Kit. ULX3S, Raspberry Pi 5, jumper wires, logic analyser.

1. Implement an SPI slave (mode 0) with a transaction format of one command byte (read or write, and the address) followed by data bytes. Bridge it to the bus from Lab 2.2.4 so every register is reachable over SPI.
2. Handle the clock domain crossing, because SPI's clock is asynchronous to the system clock. Either synchronise SCLK edges into the system domain (fine at low SPI rates) or run the shift register on SCLK and cross the finished byte with a handshake.
3. Drive it from the Raspberry Pi in Python with `spidev`: toggle an FPGA GPIO, read the ID register, program the timer.
4. Route the FPGA's interrupt line to a Pi GPIO and log it with `gpiomon`.
5. Capture a full transaction on the logic analyser and annotate it.

DONE WHEN

- [ ] Registers read and write over SPI from the Pi, the timer interrupt reaches the Pi, and the annotated capture is in the notebook.

EVIDENCE FOR THE NOTEBOOK

- RTL; the Python script; the annotated capture.

Problem set

1. Write an `always_comb` block that infers a latch by accident, and fix it. Write an `always_ff` block where a blocking assignment makes simulation differ from synthesis.
2. How many 18 kbit block RAMs does a 320 × 240 8-bit framebuffer need? A 640 × 480 4-bit one? Which fits an ECP5-85F with 208 such blocks?
3. Derive the baud counter for 115200 from 100 MHz. What is the frequency error, and how does it accumulate over a 10-bit frame? Compare with a 48 MHz clock.
4. List three ways a multi-bit signal crossing clock domains can be corrupted even when each bit passes through a two-flop synchroniser.
5. Write constraints for a 100 MHz system clock, a 12 MHz SPI clock treated as asynchronous, and a path from a configuration register that changes only at start-up. Name the constraint for the last one and say why it is legitimate.
6. A UART receiver oversamples at 16× and samples at the eighth tick. The transmitter's clock is 3 % fast. At which data bit does sampling drift into the wrong bit cell?
7. Why is a FIFO's "empty" flag safe to compute in the read domain but not the write domain? What goes wrong if "full" is computed in the read domain?
8. A path passes through 14 LUTs and a carry chain. Estimate the pipeline stages needed to bring it under 5 ns, and the throughput and latency costs.

Deliverables and stretch

Deliverables. UART, FIFO, register-map peripheral and SPI bridge, each with a testbench, and the UART and FIFO with formal proofs; the register-map document; the annotated SPI capture.

Stretch. Generate 640 × 480 video over the ULX3S's HDMI connector with a character-mode framebuffer in block RAM, writable from the bus, and check the BRAM budget against the utilisation report. Reimplement the FIFO in Amaranth and compare the generated Verilog. Build a small LiteX SoC with a VexRiscv core and look at how it generates the register map and C headers.

COMPLETION CHECKLIST

- [ ] **Lab 2.2.1:** hand-written constraints; the timing failure and both fixes annotated.
- [ ] **Lab 2.2.2:** UART echo on hardware; formal proof passing; baud tolerance measured.

- [ ] **Lab 2.2.3:** FIFO proved by induction; the binary-pointer counterexample explained.
- [ ] **Lab 2.2.4:** peripheral testbench passing; register map committed.
- [ ] **Lab 2.2.5:** SPI bridge driven from the Pi with the interrupt delivered; capture annotated.
- [ ] **Problem set:** all eight answered.
- [ ] **You can explain**, without notes: blocking versus non-blocking assignment, how an asynchronous FIFO stays correct, what slack is, and what a false path claims.

CHAPTER 2.3

Build a CPU

TIME	4 weeks
YOU BUILD	A single-cycle RV32I core passing the riscv-tests suite; a five-stage pipeline with forwarding and branch prediction; the core on the FPGA running C you compiled, driving your peripheral; the M and Zicsr extensions.
NEEDS	ULX3S, riscv32 GCC toolchain, Verilator, your Chapter 2.2 UART and peripheral.

WHAT THIS BUYS OMH

Compute is an OMH product. You will buy the CPUs, but you will size them, read their errata, argue with vendors about SMT and side channels, and tune a hypervisor around what they actually do. The only reliable way to understand a pipeline, a hazard or a branch predictor is to build one and watch it fail. After this chapter the Intel manual describes mechanisms you have built yourself.

SKIP TEST

1. Encode `addi x5, x6, -4` and `sw x7, 12(x8)` by hand from the RV32I tables.
2. Draw a five-stage pipeline and mark every forwarding path needed to run back-to-back dependent ALU instructions without stalls. Which hazard cannot be solved by forwarding?
3. A branch resolves in the EX stage. What is the misprediction penalty in cycles, and how does a 2-bit predictor change the expected penalty for a loop of 100 iterations?
4. What does a linker script's `MEMORY` block do, and why does bare-metal C need a `crt0` before `main`?
5. Why does RISC-V have no condition codes and no branch delay slots?

If all five are easy, do Labs 2.3.2 and 2.3.3 only.

Core ideas

The ISA is the contract between software and hardware. RV32I has about 40 instructions in six regular formats (R, I, S, B, U, J). Immediates are scattered across bits so that the sign bit and the register fields are always in the same place, which simplifies the decoder. Read the specification's design rationale, which explains how each choice keeps the hardware simple.

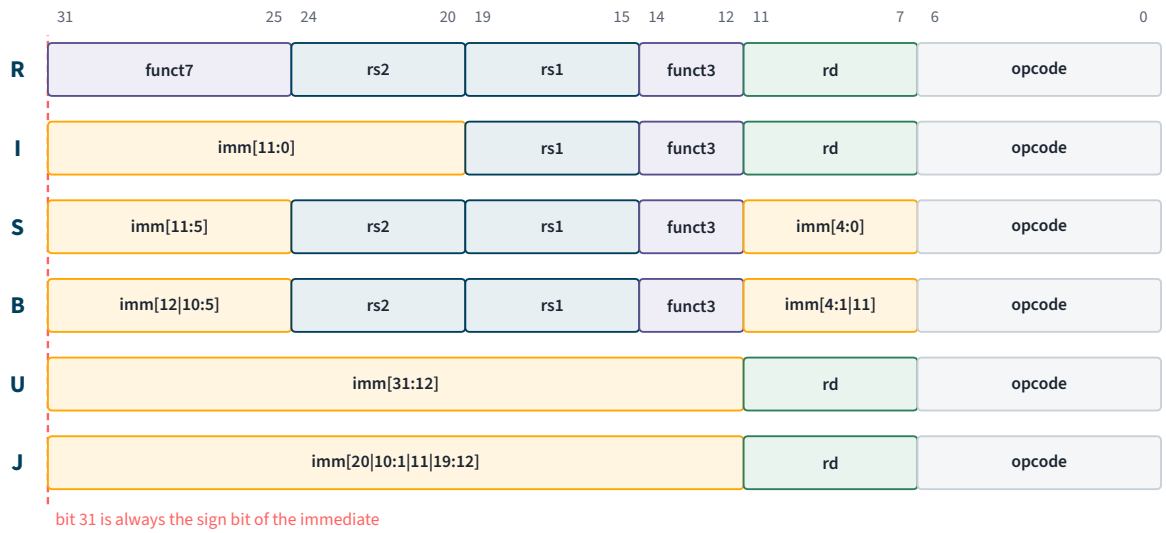


Figure 2.3.1. The six RV32I instruction formats. The register fields and the sign bit (bit 31) never move; the immediate bits are shuffled so the decoder's sign-extension and multiplexers stay small.

The datapath. The datapath fetches an instruction at PC, decodes it into control signals and register indices, reads the register file, executes in the ALU, accesses memory for loads and stores, and writes back the result. A single-cycle design does all of that in one long clock period, set by the slowest instruction (a load).

Pipelining. Split the datapath into stages separated by registers so that five instructions are in flight. Throughput rises toward one instruction per cycle, although the latency of each instruction does not fall. The period is now set by the slowest stage plus register overhead, so unbalanced stages waste time.

Hazards. A structural hazard occurs when two stages need the same resource, and separate instruction and data memories, or caches, fix it. A data hazard occurs when an instruction needs a result not yet written back. Forward it from the EX/MEM or MEM/WB registers, except when a load is followed immediately by a use, which must stall one cycle. A control hazard occurs because a branch's outcome is not known until EX, so the pipeline stalls, predicts, or both. Flushing squashes wrongly fetched instructions by turning them into no-ops.

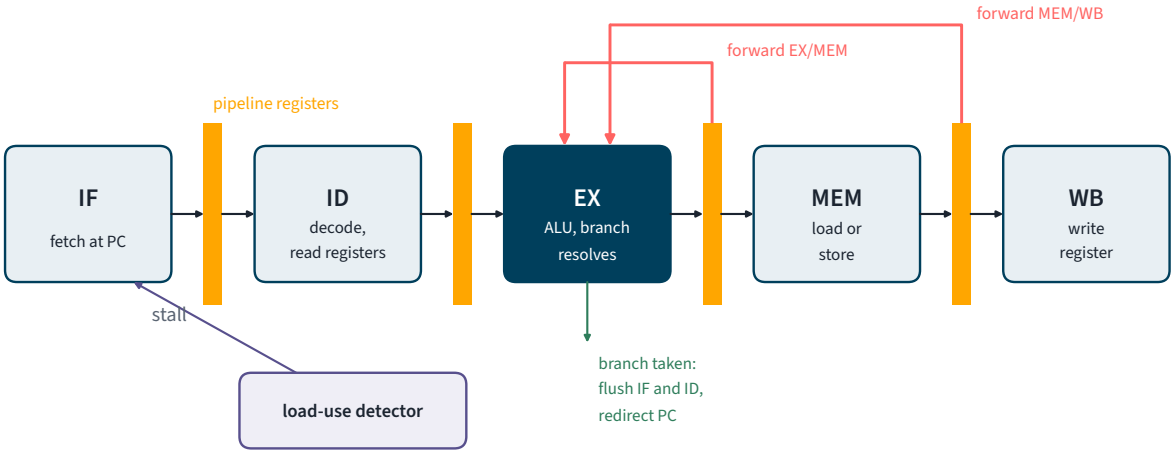


Figure 2.3.2. A five-stage pipeline with the two forwarding paths into the ALU and the load-use stall. The branch resolves in EX; a misprediction flushes the two younger instructions.

Branch prediction. Prediction is static (predict not taken, or backward taken) or dynamic: a branch history table of 2-bit saturating counters indexed by PC bits, a branch target buffer for where to go, and later global history and tournament predictors. Modern cores predict nearly every branch correctly and still lose tens of cycles when they miss, because their pipelines are deep.

Memory-mapped I/O. Peripherals live at addresses. A load from the UART status register is a bus read that the peripheral answers. Your Chapter 2.2 peripheral attaches to the data bus behind an address decoder. Loads and stores to device registers must not be cached, reordered or merged, which is why `volatile` exists in C and why the ISA has fence instructions.

Control and status registers. CSRs hold machine state: cycle counters, trap vectors, status. Zicsr gives `csrrw`, `csrrs` and `csrrc`. Chapter 2.5 puts the CSRs to use for traps and privilege.

The bare-metal toolchain. GCC targets `rv32i` with the ILP32 ABI. A linker script places `.text` in ROM, `.data` and `.bss` in RAM, and defines symbols for the stack top and section bounds. `crt0.S` sets the stack pointer, copies `.data` from ROM to RAM, zeroes `.bss` and calls `main`. `objcopy` turns the ELF into a hex file that `$readmemh` loads into block RAM. There is no OS, and no libc unless you bring one (picolibc is small).

Compliance testing. riscv-tests and riscv-arch-test are directed tests per instruction with a pass or fail signature. Passing them is your evidence that the CPU implements the ISA correctly.

Performance. Time = instructions × CPI × cycle time. Pipelining keeps CPI near 1 while shortening the clock period, and hazards raise it. Only multiple issue pushes it below 1. Measure with `mcycle` and `minstret`.

Beyond this core. Larger cores add superscalar issue, out-of-order execution with register renaming and a reorder buffer, speculation with precise exceptions, SIMD and vector units, and simultaneous multithreading. You will not build these, but you will read enough to understand a sentence such as "Spectre exploits speculative execution across a mispredicted branch". A server processor has around sixty such cores sharing a cache and a memory controller.

Reading other cores. PicoRV32 (tiny, multi-cycle), Ibex (two-stage, production quality), VexRiscv (generated, pipelined) and Rocket (Chisel, Berkeley) are worth reading after yours works.

Reading

- RISC-V Unprivileged ISA specification: chapter 1 (introduction and rationale), chapter 2 (RV32I), the M extension, Zicsr, and the instruction listing tables.
- Patterson and Hennessy, *Computer Organization and Design*, RISC-V edition: chapter 2 (skim what you know) and chapter 4 (the processor: single-cycle, pipelining, hazards, branch prediction, exceptions).
- Harris and Harris, DDCA RISC-V edition, chapter 7 (microarchitecture), as a reference once yours has a shape.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*, sections 3.1 to 3.6 (out-of-order overview; read once, do not implement).
- The GNU ld manual's "Scripts" chapter and the picolibc README.

Labs

LAB 2.3.1 Single-cycle RV32I — 3 weekends

1. Write the decoder as a table from the specification rather than from memory. Implement the register file (x0 wired to zero), the ALU, the immediate generator, the PC logic, the load/store unit with byte and halfword handling, and the control unit.
2. Implement instruction and data memory as block RAM initialised from hex, and a data bus with an address decoder placing RAM low and your peripheral and UART above it.
3. Write a Verilator testbench in C++ that loads a hex file, runs N cycles and exposes register and memory state, with a `tohost`-style mechanism for tests to report pass or fail.
4. Build riscv-tests for `rv32ui-p-*` and run every one. Fix until all pass.
5. Write `crt0.S`, a linker script and a C `main` that prints "hello" through your UART, compiled with `-march=rv32i -mabi=ilp32 -ffreestanding -nostdlib`, and watch it in simulation.

DONE WHEN

- [ ] All `rv32ui` tests pass and your C program prints through your UART in simulation.

EVIDENCE FOR THE NOTEBOOK

- Test log; `crt0.S`; linker script; the UART output.

LAB 2.3.2 A five-stage pipeline — 2 weekends

1. Insert pipeline registers between IF, ID, EX, MEM and WB. Add forwarding from EX/MEM and MEM/WB into the ALU inputs, and the load-use detector and stall.
2. Resolve branches in EX with predict-not-taken and a flush. Re-run riscv-tests.
3. Add a 2-bit predictor with a small branch target buffer. Re-run.
4. Add `mcycle` and `minstret`. Port a small benchmark (a Dhrystone-like loop, or CoreMark if it builds). Measure CPI for single-cycle-equivalent, pipelined and predicted configurations.

DONE WHEN

- [ ] Tests pass on the pipeline and the notebook has a CPI table for the three configurations with each difference explained.

EVIDENCE FOR THE NOTEBOOK

- Test logs; the CPI table.

LAB 2.3.3 On the FPGA at speed — 1 weekend

1. Synthesise for the ULX3S at 50 MHz. Find the critical path (usually the ALU or the forwarding multiplexer) and push toward the highest frequency you can close with margin.
2. Boot from block RAM with the console on the FTDI UART.
3. Run a program that configures your timer, polls for the compare match and toggles LEDs; then read a button through the GPIO input.
4. Toggle a GPIO each cycle (divided down) and confirm the real clock and `mcycle` against the scope.

DONE WHEN

- [ ] Your C runs on your CPU on the FPGA, drives your peripheral, and `mcycle` agrees with the scope.

EVIDENCE FOR THE NOTEBOOK

- The achieved frequency and its timing report; the scope capture.

LAB 2.3.4 Extensions — 1 weekend

1. Add the M extension: a multi-cycle multiplier and divider with a stall. Run the `rv32um` tests and compare benchmark CPI with and without it.
2. Implement Zicsr fully, including the immediate forms.
3. Add `fence` and `fence.i` as no-ops, with a comment saying what each must do once there are caches. Chapter 2.4 makes them real.

DONE WHEN

- [ ] `rv32um` and the CSR tests pass.

EVIDENCE FOR THE NOTEBOOK
- Test logs; the CPI comparison.

LAB 2.3.5 Read three other cores — 4 h

Read Ibex's pipeline, VexRiscv's generated Verilog for a similar configuration, and PicoRV32. Write one page: where your design is the same, where it differs, and three things you would change having read theirs. Note how Ibex keeps exceptions precise in a pipeline, because Chapter 2.5 needs it.

DONE WHEN
- ☐ The page is committed.

Problem set

1. Hand-encode `lui x1, 0x12345`, `jal x0, -16`, `bne x2, x3, +32` and `sb x4, -1(x5)`. Decode `0x00A28293` and `0xFE0508E3`.
2. Stage delays are IF 300 ps, ID 200 ps, EX 350 ps, MEM 400 ps and WB 150 ps, and pipeline registers cost 50 ps. Compute clock period and per-instruction latency for single-cycle and pipelined designs, and the speedup for a long program with no hazards.
3. A program is 25 % loads and 20 % branches; 40 % of loads are followed by a dependent instruction; branches mispredict 15 % of the time with a 2-cycle penalty. Compute CPI on your pipeline.
4. Write the forwarding logic as a truth table over the EX instruction's source register and the MEM and WB destination registers, including the priority when both match.
5. Why must the load-use hazard stall rather than forward? Draw the timing.
6. Explain why RISC-V puts the sign bit at bit 31 in every format and what that saves in the decoder.
7. A predictor uses 10 PC bits to index 1,024 2-bit counters and two hot branches alias. Describe the interference and one mitigation.
8. A workload spends 30 % of its time in a routine the M extension makes four times faster. What is the overall speedup? What if loads also become 20 % faster?
9. Write a linker-script fragment placing `.text` at 0x0000_0000 (64 KiB of ROM) and `.data` and `.bss` at 0x1000_0000 (32 KiB of RAM), with the stack at the top of RAM and the symbols `crt0` needs.
10. Read the Spectre v1 paper's abstract and explain, in terms of your pipeline plus a hypothetical cache, what speculation would have to do for the attack to work. Does your core speculate?

Deliverables and stretch

Deliverables. A pipelined RV32IM core passing riscv-tests, running on the FPGA at a documented frequency, with the CPI table; `crt0.S`, the linker script and a small runtime reused in Chapters 2.4 and 2.5; the comparison page from Lab 2.3.5.

Stretch. Replace predict-not-taken with gshare and measure. Add a two-issue in-order front end for ALU pairs. Add the C extension and measure code size and CPI. Run riscv-arch-test instead of riscv-tests and compare coverage.

COMPLETION CHECKLIST
- ☐ **Lab 2.3.1:** all `rv32ui` tests passing on the single-cycle core; C prints through your UART.
- ☐ **Lab 2.3.2:** the pipelined core passes; CPI table for three configurations.
- ☐ **Lab 2.3.3:** the core runs on the ULX3S at a recorded frequency; `mcycle` checked against the scope.

- [ ] **Lab 2.3.4:** `rv32um` and CSR tests pass.
- [ ] **Lab 2.3.5:** the one-page comparison committed.
- [ ] **Problem set:** all ten answered.
- [ ] **You can explain**, without notes: the five stages, each hazard type and its fix, how a 2-bit predictor works, and what `crt0` does before `main`.

CHAPTER 2.4

Memory and interconnect

TIME	3 weeks
YOU BUILD	Caches on your CPU with measured miss rates; your CPU running from external SDRAM; C++ experiments that reveal a real machine's cache sizes, line size, page size and memory model; a PCIe device's configuration space decoded by hand and its registers read with no driver.
NEEDS	ULX3S with its SDRAM, the lab server with a PCIe NIC, the workstation, the Raspberry Pi 5 for an Arm comparison.

WHAT THIS BUYS OMH

OMH's specification promises Ethernet and CXL fabrics and erasure coding across hot-swap bays, and both claims rest on this chapter. CXL is a coherent memory protocol over PCIe, and the drive bays are PCIe endpoints. The erasure-coding budget is bounded by memory bandwidth and by how many times data crosses the interconnect. An IOMMU lets Compute hand a NIC to a tenant VM safely, and ECC memory makes a storage appliance trustworthy. To talk to vendors as a peer you must know what a page walk, a TLP and a MESI transition cost.

SKIP TEST

1. A 32 KiB, 4-way set-associative cache with 64-byte lines on a 32-bit address: how many bits of index, tag and offset?
2. Walk a Sv39 virtual address through three levels of page table. How many memory accesses does a TLB miss cost, and why do huge pages help?
3. Two cores each write a different variable in the same cache line. What does MESI do on each write, and why is it slow?
4. Write the x86 store-buffer litmus test and say which outcomes are allowed. Which C++ `memory_order` forbids the surprising one?
5. A PCIe device raises an interrupt and DMAs 4 KiB into host memory. Describe the TLPs involved and where the IOMMU intervenes.

If all five are easy, do Labs 2.4.1 and 2.4.4 only.

Core ideas

SRAM and DRAM. SRAM (Chapter 2.1's 6T cell) is fast and static, and caches are built from it. DRAM stores a bit as charge on a capacitor behind one transistor. It is dense and cheap and must be refreshed every 64 ms because the charge leaks. Reading destroys a row, so the row is copied into the sense amplifiers (the row buffer), served from there, and written back.

DRAM organisation. DRAM is organised into channels, DIMMs, ranks, chips, banks, rows and columns. A row hit is fast, and a row miss pays for a precharge and an activate. Timing parameters (tRCD, tCAS, tRP, tRAS) are counted in DRAM clocks and are what a memory controller schedules around. DDR transfers on both edges. DDR4 and DDR5 add bank groups and speed, and DDR5 adds on-die ECC. The controller must train the interface at boot to align signals, which is why memory initialisation is the slowest and most fragile part of firmware.

ECC. Single-error-correct, double-error-detect Hamming codes cover 64 data bits with 8 check bits, and chipkill or SDDC survives a whole chip failing. Servers use it because at scale memory errors happen daily, and a storage appliance that silently corrupts is worse than one that fails.

The hierarchy and locality. In the sequence registers, L1, L2, L3, DRAM, NVMe and network, each level is about an order of magnitude slower and larger than the one before. Programs show temporal and spatial locality, which is why caches work.

Cache organisation. A cache is organised in lines (64 bytes almost everywhere), sets and ways. A direct-mapped cache has one way, and a fully associative cache has one set. An address splits into tag, index and offset. A cache is write-through or write-back, allocates on writes or does not, and replaces lines by LRU, pseudo-LRU or random choice. Misses are compulsory, capacity or conflict. Average memory access time is hit time plus miss rate times miss penalty. Prefetchers guess the next line.

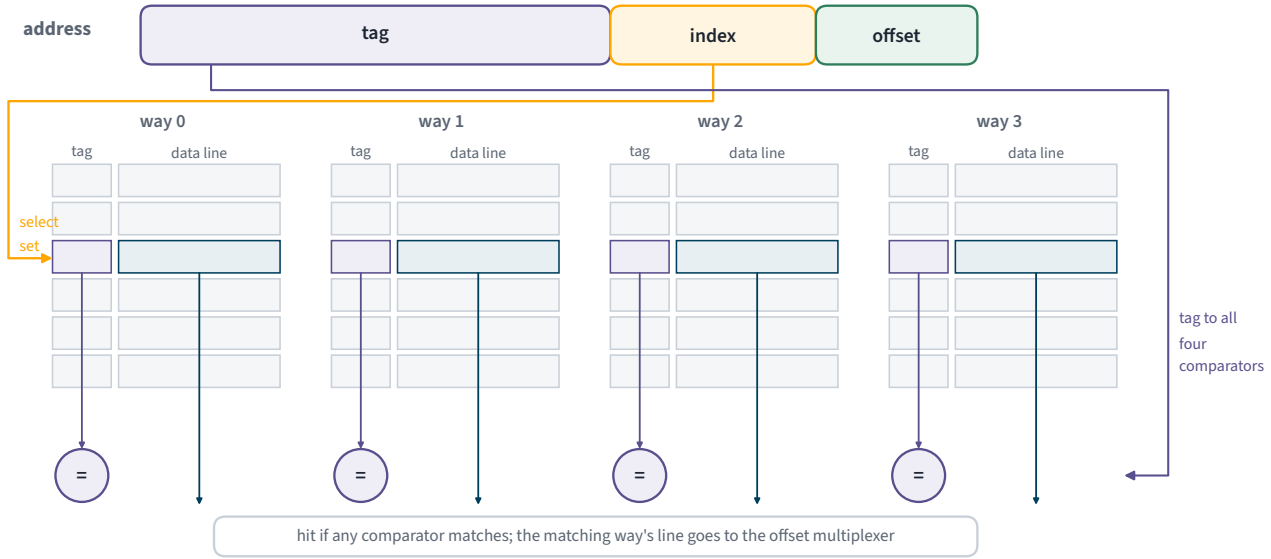


Figure 2.4.1. A 4-way set-associative cache. The index selects a set, the four tags are compared in parallel, and the offset selects bytes within the hitting line.

Coherence. Several caches holding one line must agree. Snooping protocols broadcast, and directory protocols track owners. The common protocols are MESI (Modified, Exclusive, Shared, Invalid) and MOESI, which adds Owned. A write to a Shared line invalidates every other copy, so two cores writing different words of one line pass it back and forth, which is called false sharing. Coherence concerns a single location, and consistency concerns the ordering of accesses across locations.

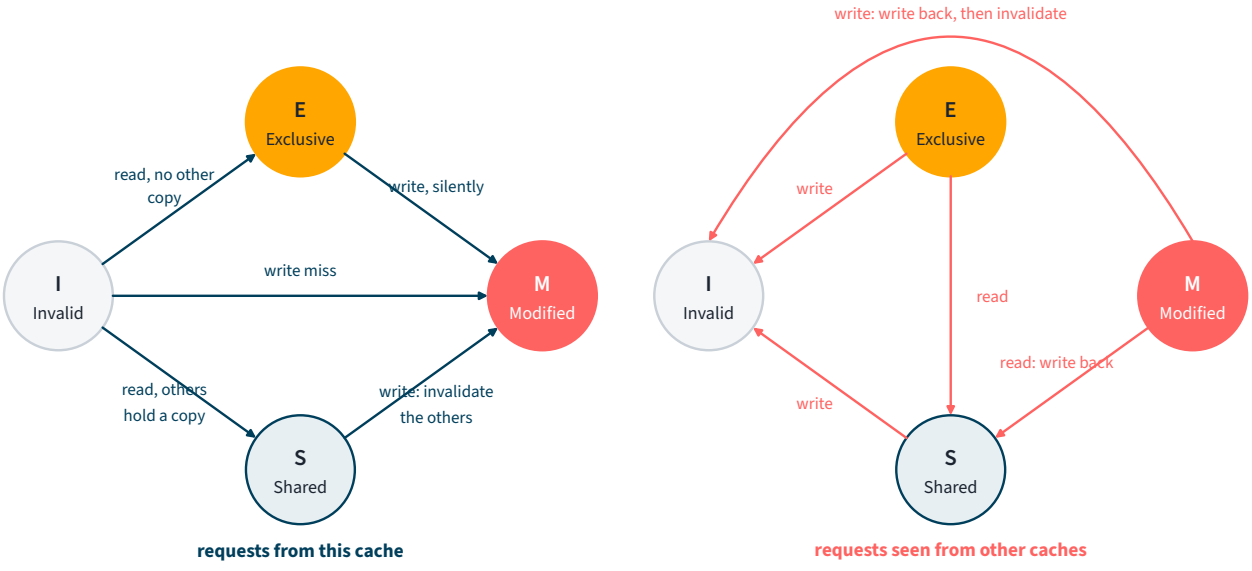


Figure 2.4.2. MESI from one cache's point of view. Left, the cache's own reads and writes; right, other caches' requests, seen on the bus or from the directory, which force write-backs and invalidations. Read hits leave the state unchanged.

Consistency and your C++ atomics. Programmers assume sequential consistency, but hardware provides weaker models. x86 is TSO: stores may wait in a store buffer behind later loads, and nothing else reorders. Arm and RISC-V (RVWMO) are weaker, and loads and stores reorder freely unless fenced. C++ acquire and release map to cheap instructions on x86 and to barriers or load-acquire and store-release instructions on Arm. Litmus tests (store buffering, message passing, load buffering) are how you reason about what a model allows.

Virtual memory. Memory is mapped in pages (4 KiB by default) through multi-level page tables indexed by virtual-address bits, whose entries hold a physical page number with valid, permission, accessed and dirty bits. The TLB caches translations, and a miss walks the table at one memory access per level (three for Sv39, four for x86-64). Huge pages (2 MiB, 1 GiB) cut walks and TLB pressure. Address-space identifiers let the TLB hold several processes' entries.

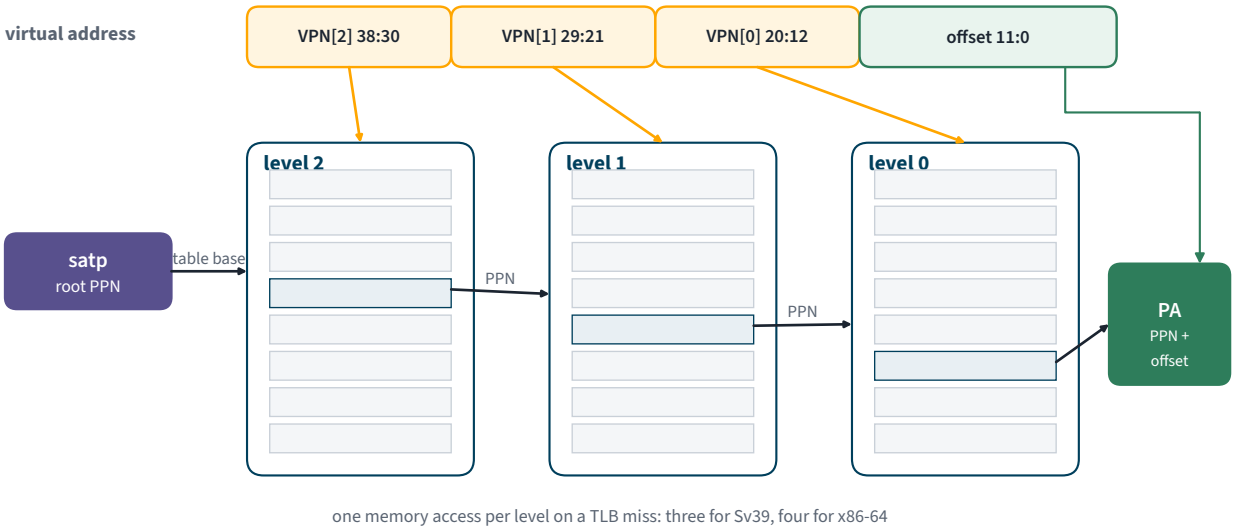


Figure 2.4.3. A Sv39 page walk. Three 9-bit fields index three levels of 512-entry tables starting from the root in `satp`; the final entry's physical page number joins the 12-bit offset.

The IOMMU. The IOMMU applies the same idea to devices. A device's DMA address is translated through I/O page tables, so a device can touch only memory its owner mapped for it. Implementations include Intel VT-d, AMD-Vi, Arm SMMU and the RISC-V IOMMU. Without one, PCIe passthrough to a VM is unsafe. With one, VFIO becomes possible and a buggy NIC cannot overwrite kernel memory.

On-chip interconnect. Wishbone and AXI connect a few blocks, and crossbars and networks-on-chip connect many. Bandwidth is width times clock, and contention and arbitration add latency. Little's law relates them: concurrency = throughput × latency. A core with 10 outstanding misses at 100 ns each sustains one line per 10 ns, which bounds its memory bandwidth.

PCIe. PCIe uses point-to-point serial lanes (×1 to ×16) and a layered protocol: physical (128b/130b encoding at Gen3, PAM4 at Gen6), data link (ACK/NAK and flow-control credits) and transaction (TLPs: memory read and write, configuration, message). Devices are enumerated by walking buses and reading configuration space: vendor and device ID, class, BARs and capabilities. BARs say how much MMIO a device needs, and firmware or the OS assigns addresses. Interrupts are message-signalled writes (MSI, MSI-X with per-vector addresses). DMA is the device issuing memory TLPs itself. A server must handle Advanced Error Reporting, hot-plug, power states and Access Control Services, which a laptop can ignore.

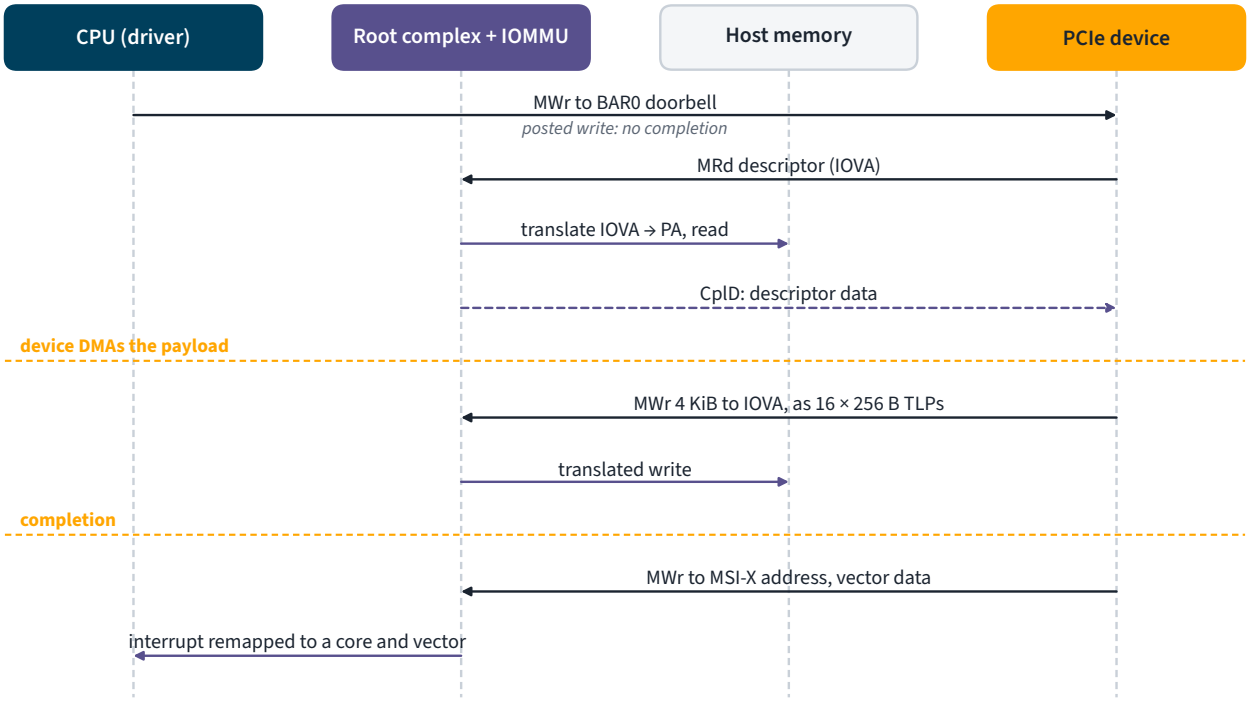


Figure 2.4.4. A device's DMA write and MSI-X interrupt as TLPs. Both pass through the root complex, where the IOMMU translates the DMA address and remaps the interrupt.

CXL. Compute Express Link reuses PCIe's physical layer for three protocols: CXL.io (PCIe as usual), CXL.cache (a device caches host memory coherently) and CXL.mem (the host uses device memory as its own). Type 1 devices are caching accelerators, Type 3 are memory expanders and pools, Type 2 do both. CXL 2.0 adds switches and pooling, and 3.x adds fabrics and shared memory. For OMH, CXL means memory attached to, pooled across and moved between modules, and a coherent path for accelerators.

NUMA. A NUMA machine has several sockets or chiplets, each with local memory, and remote access is slower. Thread and memory placement matter, and a hypervisor must expose or hide topology deliberately.

NVMe, previewed. NVMe places submission and completion queues in host memory and doorbells in a BAR, with commands carrying physical addresses and completions signalled by MSI-X. It depends on everything above. You write the driver in Chapter 4.1.

Reading

- Patterson and Hennessy, CO&D RISC-V edition, chapter 5 (all).
- Drepper, *What Every Programmer Should Know About Memory*, parts 2 to 6.
- Nagarajan, Sorin, Hill and Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., chapters 1 to 4 and 6 to 8 (free).
- Hennessy and Patterson, CA:AQA, chapter 2 and appendix B.
- Jackson and Budruk, *PCI Express Technology 3.0*: chapters 1 to 6, 8 to 10 and 17.
- CXL 3.1 specification, chapters 1 and 2; JEDEC DDR4 standard sections 2 to 4; Intel VT-d specification chapters 1 to 3.

Labs

LAB 2.4.1 Caches on your CPU — 2 weekends

1. Add a direct-mapped instruction cache and a write-back data cache (4 KiB each, 32-byte lines). Peripheral addresses bypass the cache. Make `fence` and `fence.i` real: `fence` drains the write buffer, and `fence.i` also writes back dirty data-cache lines and invalidates the I-cache.
2. Simulate memory-bound benchmarks (large-stride array sum, pointer chase, matrix multiply) with a memory model of configurable latency. Expose hit and miss counters as CSRs; record CPI and miss rate.
3. Change to 2-way with pseudo-LRU, then to 64-byte lines, then add a next-line I-cache prefetcher, measuring each.

DONE WHEN

- [ ] riscv-tests still pass with caches on, and a table of CPI and miss rate across at least four configurations explains each change.

EVIDENCE FOR THE NOTEBOOK

- The configuration table and the benchmark sources.

LAB 2.4.2 External SDRAM — 1 weekend

1. Read the datasheet of the ULX3S's SDRAM: state diagram, initialisation sequence and timings. Write the init sequence and command timings down before looking at any controller.
2. Integrate an open controller (LiteDRAM or a ULX3S example) behind your cache and run a program from SDRAM. If time allows, write your own simple controller: init, refresh timer, single-word read and write with row open and close.
3. With a behavioural SDRAM model, count row hits and misses for sequential and strided access, and relate them to your line size.

DONE WHEN

- [ ] C runs from SDRAM on the FPGA and you can explain the init sequence and what refresh costs in bandwidth.

EVIDENCE FOR THE NOTEBOOK

- The written init sequence; the row-hit counts.

LAB 2.4.3 Memory experiments on a real machine — 1 weekend

Kit. Workstation, lab server, Raspberry Pi 5.

1. Stride benchmark: touch one byte every S bytes across a large array; plot time per access against S and read off the line and page sizes.
2. Working-set benchmark: random access within N bytes; plot against N and read off L1, L2 and L3. Compare with `lscpu`.
3. False sharing: two threads incrementing adjacent `std::atomic<int>`s against padded ones, measured with `perf stat -e cache-misses,instructions,cycles`.
4. TLB: pointer-chase 4 GiB with 4 KiB pages against 2 MiB huge pages; measure `dTLB-load-misses`.
5. Store-buffer litmus test: two threads on x86, no fences, a million runs, count "both read zero"; add a sequentially consistent fence and repeat. On the Pi, run the message-passing test with relaxed atomics and observe reordering x86 never shows.
6. On the server, `numactl --membind` remote against local for a bandwidth benchmark; record the ratio.

DONE WHEN

- [ ] Plots show the line size, page size and three cache levels, and the litmus tests demonstrate TSO on x86 and weaker ordering on Arm.

EVIDENCE FOR THE NOTEBOOK

- C++ sources, plots and the litmus counts.

LAB 2.4.4 PCIe by hand

1 weekend

Kit. Lab server with the NIC installed.

1. Read `lspci -vvv -s <nic>` slowly: vendor and device ID, class, each BAR's size and type, the capability list, MSI-X with its table BAR and offset, link speed and width, and AER.
2. Without `lspci`, dump the 256-byte standard configuration space from sysfs and decode every field from the PCI specification's header layout. Walk the capability list to MSI-X. Determine BAR sizes by the write-all-ones method, with the driver unbound.
3. Unbind the driver, `mmap` `resource0` from sysfs in a C program and read harmless registers (the 82599 datasheet lists device status and ID registers), so that you talk to the PCIe device with no driver at all.
4. Rebind, read `/proc/interrupts` for the MSI-X vectors and their affinity, and watch counts move under traffic.
5. Find the IOMMU groups and explain why the NIC is or is not alone in its group, and what that means for passthrough.

DONE WHEN

- [ ] The notebook has a hand-decoded configuration space with every field labelled, and your program's register reads match `lspci`.

EVIDENCE FOR THE NOTEBOOK

- The annotated hex dump; the C program and its output.

LAB 2.4.5 A page-table walker

1 weekend

Add a Sv32 page-table walker and a small fully associative TLB to your CPU's instruction and data paths, gated by a `satp`-like CSR. Test with a program that builds a page table, enables translation, and checks that a load through a mapped page works and one through an unmapped page reports a fault. Fault handling arrives in Chapter 2.5. If you are behind, skip this lab now and build it before Chapter 2.5, which needs it.

DONE WHEN

- [ ] Translated loads work and an unmapped access raises a fault signal in simulation.

Problem set

1. A 64 KiB, 8-way, 64-byte-line cache with 48-bit physical addresses: index, tag and offset widths, and total tag storage.
2. AMAT with L1 hits of 1 ns at 95 %, L2 hits of 4 ns on 80 % of L1 misses, and 80 ns memory. Then add an L3 at 12 ns hitting 60 % of L2 misses.
3. A DDR4-3200 channel is 64 bits wide. Peak bandwidth? With 8 channels? Why does measured bandwidth reach about 80 % and a single core far less (Little's law with 10 outstanding misses at 90 ns)?

4. Refresh: 8,192 refresh commands every 64 ms at 350 ns each. What fraction of time is the bank unavailable, and what happens at 16 Gbit densities?
5. Construct the SECDED code for 8 data bits and show it corrects one flipped bit and detects two.
6. Walk virtual address 0x0000_003F_C000_1234 through Sv39 with `satp` pointing at 0x8000_0000, giving the byte offset into each level's table.
7. PCIe Gen4 ×4 runs at 16 GT/s per lane. Compute usable bandwidth after 128b/130b, then after TLP overhead with 256-byte payloads and 20 bytes of header and framing. Compare with a drive advertising 7,000 MB/s.
8. Give the MESI state of a line in cores A and B after: A reads, B reads, A writes, B reads, B writes, A reads. Count bus transactions.
9. Explain why the store-buffer test can print 0,0 on x86 and the message-passing test cannot, then what changes on RISC-V RVWMO.
10. A CXL Type 3 expander adds 512 GiB at 250 ns against 90 ns local. For a key-value store with 5 % of accesses going to the far tier, what is the effective access time, and which OMH module does that trade suit?

Deliverables and stretch

Deliverables. The cached CPU passing tests and running from SDRAM, with the configuration table; the C++ measurement programs and plots; the hand-decoded configuration space and the register-reading program.

Stretch. Build a two-core version of your CPU with snooping MSI coherence on a shared bus and a spin-lock test. Write a `perf` script estimating the cost of a page walk on the server by comparing 4 KiB and 1 GiB pages. Draw the CXL.mem request and response flow for a host read from a Type 3 device.

COMPLETION CHECKLIST

- ☐ **Lab 2.4.1:** caches with fences working; four-configuration table explained.
- ☐ **Lab 2.4.2:** C runs from SDRAM; init sequence and refresh cost written down.
- ☐ **Lab 2.4.3:** plots revealing line, page and cache sizes; litmus results on x86 and Arm; the NUMA ratio.
- ☐ **Lab 2.4.4:** configuration space decoded by hand; driver-less register reads matching `lspci`.
- ☐ **Lab 2.4.5:** the page-table walker, here or before Chapter 2.5.
- ☐ **Problem set:** all ten answered.
- ☐ **You can explain**, without notes: tag, index and offset; MESI and false sharing; TSO versus RVWMO; a page walk; what a TLP is and where the IOMMU sits.

CHAPTER 2.5

Privilege and the platform

TIME	2 weeks
YOU BUILD	Machine and user modes, precise traps, a timer interrupt, PMP and a two-call system-call ABI on your own CPU; preemptive multitasking on the FPGA; an x86-64 bare-metal program with its own GDT, IDT, APIC timer and a ring-3 system call, on QEMU and on the server; annotated ACPI and SMBIOS tables from the server.
NEEDS	Your CPU, QEMU, the lab server with serial-over-LAN.

WHAT THIS BUYS OMH

A hypervisor uses privilege levels to present another program with a machine that differs from the real one. A kernel uses them to share the machine between programs. Measured boot is a chain of privilege handoffs, each measured before the next. None of Chapters 2.7, 4.1 or 4.4 can be understood without knowing what a trap is, what state it saves, and who may touch which register. This chapter makes that concrete on your own CPU, then shows the same shape on x86 and Arm, the platforms OMH will ship.

SKIP TEST

1. On RISC-V, an `ecall` from U-mode arrives in M-mode. List every CSR the hardware writes, and what the handler must do before it can safely use a stack.
2. What does `medeleg` do, and why does a hypervisor care?
3. On x86-64 a user process executes `syscall`. Which registers change, where does the kernel stack come from, and why does the kernel still need a TSS in long mode?
4. What is the difference between the local APIC and the I/O APIC, and how does an MSI from a PCIe device reach a particular core?
5. Why does an OS need ACPI on x86 but a device tree on most Arm boards, and what breaks if both are absent?

If all five are easy, do Lab 2.5.1 only.

Core ideas

Why privilege exists. Some operations must be reserved: changing address translation, masking interrupts, touching devices. A privilege level is a bit or two of state the hardware consults before allowing them, and a trap is the controlled transfer that happens when a less privileged program needs one. Everything else is mechanism for making that transfer fast, safe and complete.

RISC-V privilege. RISC-V has three modes: Machine (always present, highest), Supervisor (for kernels) and User. The M-mode CSRs are `mstatus` (global interrupt enable and previous privilege), `mtvec` (trap vector), `mepc` (return address), `mcause` (why), `mtval` (bad address or instruction), `mscratch` (a free register for the handler) and `mie` / `mip` (interrupt enable and pending), with S-mode twins. Traps are synchronous exceptions (illegal instruction, misaligned access, page fault, `ecall`) or asynchronous interrupts (software, timer, external). All go to M-mode unless delegated with `medeleg` and `mideleg`, which is how a kernel in S-mode gets its own traps and how a hypervisor decides what a guest may handle. `mret` and `sret` return.

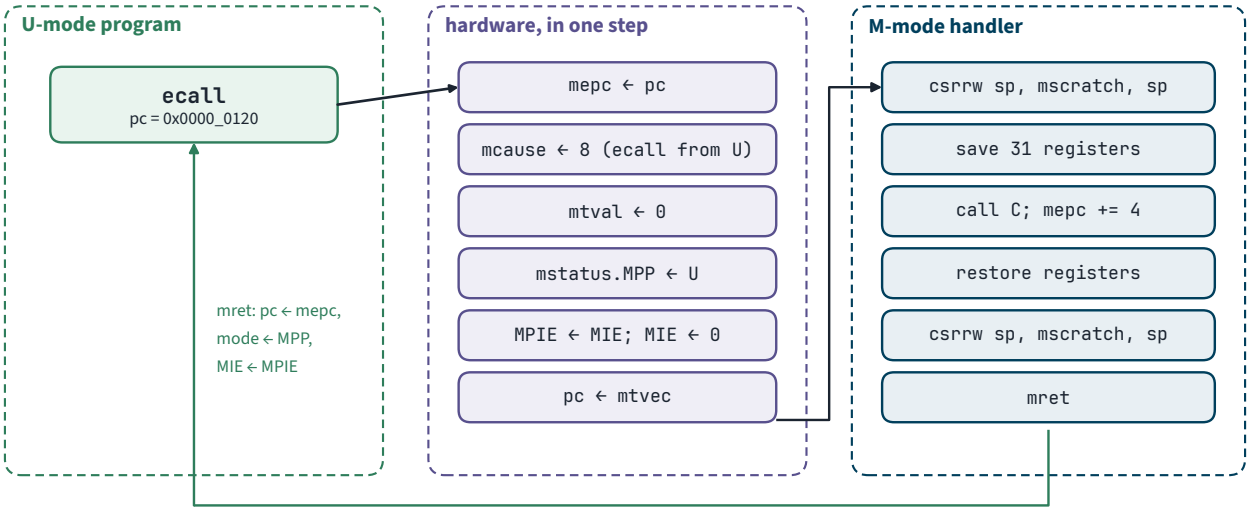


Figure 2.5.1. A trap from U-mode to M-mode on RISC-V. Hardware saves only the PC, the cause and the previous mode; everything else, starting with a usable stack from `mscratch`, is the handler's job.

Timers and interrupt controllers. The CLINT provides `mtime`, `mtimecmp` and software interrupts per hart. The PLIC aggregates device lines with priorities and per-hart enables, and AIA adds message-signalled interrupts. Every preemptive scheduler is driven by the timer interrupt.

Physical Memory Protection. PMP registers fence address ranges off from lower modes even without paging. Firmware uses PMP to protect itself from the kernel on simple cores.

Paging on RISC-V. `satp` holds the mode (Sv32, Sv39, Sv48) and the root table. Entries carry R, W, X, U, G, A and D bits. `sfence.vma` flushes the TLB. Supervisor code may not touch U pages unless `sstatus.SUM` is set, which catches a kernel dereferencing a user pointer by accident.

x86-64 equivalents. x86-64 uses rings 0 and 3. Long mode makes segmentation vestigial, but the GDT must still hold code and data descriptors and a TSS descriptor. The IDT holds 256 gates: exceptions 0 to 31, then device interrupts. An interrupt pushes SS, RSP, RFLAGS, CS and RIP (plus an error code for some exceptions), and when privilege changes the new stack comes from the TSS's RSP0, or from the Interrupt Stack Table for critical exceptions such as double fault. CR0, CR3, CR4 and EFER control protection, paging and long mode, and model-specific registers configure the rest (`STAR` and `LSTAR` for `syscall`, the APIC base, later VMX). `syscall` does not switch stacks, so the kernel does it with `swapgs` and a per-CPU pointer. Paging has four levels with 4 KiB, 2 MiB and 1 GiB pages, and PCIDs tag the TLB.

x86 interrupts. Each core's local APIC holds a timer, local vectors and inter-processor interrupts, and every interrupt for that core is delivered to it. The I/O APIC routes legacy wired interrupts. MSI and MSI-X let a device write directly to a local APIC's address with a vector number, which is why modern servers barely use the I/O APIC. x2APIC uses MSRs instead of MMIO.

Legacy and hidden modes. The CPU starts in real mode at reset and is moved to protected mode and then long mode, each with its own setup sequence (Chapter 2.7 does them). System Management Mode is a hidden, more privileged mode entered by an SMI and invisible to the OS and the hypervisor. Firmware uses it for power management, and attackers target it. Microcode updates are loaded at boot.

Armv8-A. Arm has four exception levels: EL0 (user), EL1 (kernel), EL2 (hypervisor) and EL3 (secure monitor, where Trusted Firmware-A runs). Each level has its own vector table, indexed by exception type and source. TTBR0 and TTBR1 split user and kernel translation, and EL2 owns a second stage. The GICv3 has a distributor, redistributors and CPU interfaces, with an ITS for message-signalled interrupts. PSCI powers cores on and off. The Raspberry Pi 5 follows this model, except that its interrupt controller is a GIC-400 (GICv2), which has no redistributors or ITS.

Platform description. A kernel cannot discover the interrupt controller, the timer, the core count or non-enumerable buses. On x86, firmware provides ACPI tables: RSDP, XSDT, FADT, MADT (APICs and cores), SRAT (NUMA), MCFG (PCIe configuration space), and DSDT and SSDTs containing AML bytecode the kernel

interprets. On embedded Arm and RISC-V a device tree does the same job statically: nodes with `compatible`, `reg` and `interrupts`, compiled to a blob the bootloader hands over. Arm servers use ACPI too.

SMBIOS. SMBIOS tables describe the platform's identity: manufacturer, serial number, DIMMs and slots. `dmidecode` reads them. BMCs and inventory systems depend on them, and OMH's fleet identity starts there before attestation takes over.

Reading

- RISC-V Privileged specification: chapter 1, chapter 3 (machine level, all), chapter 4 (supervisor level), and the Sv32 and Sv39 sections.
- Intel SDM volume 3A: chapters 2 to 6, the TSS parts of chapter 8, and the APIC chapters.
- OSDev wiki: GDT, IDT, Exceptions, APIC, IOAPIC, Paging, Setting Up Long Mode.
- ACPI specification chapter 5 (system description tables) and chapter 6 (`_CRS`, `_PRT`); skim the ASL reference.
- Arm's Cortex-A programmer's guide for Armv8-A: fundamentals, AArch64 exception handling, memory management. The Devicetree specification, chapters 2 and 3.

Labs

LAB 2.5.1 M-mode on your CPU — 2 weekends

1. Add `mstatus` (MIE, MPIE, MPP), `mtvec`, `mepc`, `mcause`, `mtval`, `mscratch`, `mie` and `mip`, and add `ecall`, `ebreak` and `mret`. On any exception or enabled interrupt, save PC to `mepc`, set `mcause`, disable interrupts and jump to `mtvec`. Make exceptions precise: instructions after the faulting one must have no side effects.
2. Add a CLINT-style `mtime` / `mtimecmp` peripheral and wire the timer interrupt.
3. Write an assembly trap handler that saves all registers to a trap frame on a dedicated stack found through `mscratch`, calls a C dispatcher, restores and `mret`s. The dispatcher prints the cause and handles it or panics.
4. Test illegal instruction, misaligned load, `ecall`, and a 10 ms timer tick.
5. Run two bare-metal tasks with their own stacks, with the timer handler swapping saved frames. Watch two loops interleave on the UART.

DONE WHEN

- [ ] All four traps are handled with the correct `mcause` and `mtval`, and two tasks alternate under timer preemption on the FPGA.

EVIDENCE FOR THE NOTEBOOK

- The handler source; UART logs of each trap and of the interleaved tasks.

LAB 2.5.2 U-mode and protection on your CPU — 1 weekend

1. Add U-mode: `mret` with `MPP = 0` enters it; CSR access and `mret` in U-mode trap as illegal.
2. Implement PMP with at least two NAPOT entries. Make M-mode code and data inaccessible from U. A user program reads the timer (allowed), then writes a protected address (traps); handle it and kill the task.
3. If you built the page-table walker in Lab 2.4.5, enable Sv32 for the user task, leave the kernel unmapped, and show a page fault with `mtval` holding the address.
4. Implement a two-call ABI over `ecall`: `write(fd, buf, len)` and `exit(code)`. The user program prints through it.

DONE WHEN

- [ ] A U-mode program prints through `ecall` and is killed cleanly when it touches protected memory.

EVIDENCE FOR THE NOTEBOOK

- The ABI document; the trap log of the kill.

LAB 2.5.3 x86-64 bare metal in QEMU and on the server

2 weekends

Use a Multiboot2 stub loaded by GRUB, or the Limine bootloader, so this lab is about the CPU; Chapter 2.7 replaces the stub with your own loader.

1. Set up a GDT with 64-bit code and data descriptors and a TSS with RSP0 and one IST entry. Set up an IDT whose 32 exception handlers dump the saved frame to the 16550 at 0x3F8.
2. Trigger `#DE`, `#UD`, `#PF` (read CR2) and `#DF` via a stack overflow into an unmapped guard page, and confirm the IST stack saved you.
3. Enable the local APIC (x2APIC if offered), calibrate its timer against the PIT or TSC, and print a tick every 10 ms with a proper EOI.
4. Set `LSTAR`, `STAR` and `SFMASK`, enable `EFER.SCE`, enter ring 3 via `iretq`, execute `syscall` and return with `sysretq`, swapping stacks with `swapgs` and a per-CPU structure in GS.
5. Run the same binary on the server through a GRUB entry with the serial console over SOL, and note what differs.

DONE WHEN

- [ ] The four exceptions print correctly, the APIC timer ticks, and a ring-3 system call round-trips on QEMU and on the server.

EVIDENCE FOR THE NOTEBOOK

- Serial logs from both machines.

LAB 2.5.4 Read your server's ACPI and SMBIOS

5 h

1. Extract and disassemble the DSDT and SSDTs with `acpidump`, `acpixtract` and `iasl -d`. Find the PCI root bridge `_CRS` and check its MMIO windows against `lspci`'s bridge resources. Find the `_PRT` and trace how a legacy interrupt on your NIC's slot would route.
2. Parse the MADT by hand from the hex: count local APICs, I/O APICs and interrupt source overrides; compare with `lscpu` and `/proc/interrupts`.
3. From the MCFG compute the address of your NIC's extended configuration space and confirm it against Lab 2.4.4.
4. Draw the NUMA distance table from the SRAT and SLIT if present.
5. With `dmidecode`, record the system and board serials and each DIMM's part number and speed; note which the BMC exposes over Redfish (Chapter 2.7).

DONE WHEN

- [ ] The annotated MADT, the cross-checked root-bridge resources and the verified extended-configuration address are in the notebook.

EVIDENCE FOR THE NOTEBOOK

- Annotated hex and ASL excerpts.

Problem set

1. Trace an `ecall` from U-mode on RISC-V: which CSRs change, in what order the handler must save state, and where the return address comes from. Repeat for a timer interrupt arriving in M-mode with interrupts enabled.
2. Why does the handler need `mscratch` (or `swapgs` on x86)? Describe the bug you get without it when a user program sets SP to garbage before `ecall`.
3. What does delegating the timer interrupt to S-mode change about who runs when? Why would a hypervisor delegate page faults but not external interrupts?
4. On x86-64, a `#PF` occurs in ring 3. List every value pushed, where the stack comes from and what CR2 holds. A `#DF` then occurs on a corrupted stack: what saves you?
5. Compute the Sv39 walk for VA 0xFFFF_FFC0_0001_0000 and explain why the address must be sign-extended from bit 38.
6. Compare the path of a PCIe MSI to a handler on x86 (device write, local APIC, vector, IDT) and on Arm (device write, GIC ITS, LPI, redistributor, vector table). Where does each store the vector-to-handler mapping?
7. Why is SMM a problem for a hypervisor's security model? What can measured boot do about it, and what can it not?
8. Write the device-tree node for your timer at 0x4000_0000 with an interrupt on PLIC source 3, then the equivalent ACPI `Device` with `_HID` and `_CRS`. What does a driver key on in each?
9. What goes wrong if two cores take the timer interrupt through one shared trap stack? Fix it.

Deliverables and stretch

Deliverables. Your CPU with M and U modes, PMP, traps, a timer, preemptive tasks and a two-call ABI; the x86-64 bare-metal program running on QEMU and the server; annotated ACPI and SMBIOS notes.

Stretch. Add S-mode with delegation and run a tiny S-mode kernel under your M-mode firmware, which is how OpenSBI is structured. Port Lab 2.5.3 to the Pi 5 at EL1 with vector tables, the generic timer and the GIC. Mark which CSRs from Lab 2.5.1 acquire virtual twins in the RISC-V Hypervisor extension.

COMPLETION CHECKLIST

- ☐ **Lab 2.5.1:** precise traps with correct causes; timer preemption of two tasks on the FPGA.
- ☐ **Lab 2.5.2:** U-mode program using your ABI and killed on a PMP violation.
- ☐ **Lab 2.5.3:** exceptions, APIC timer and ring-3 system call on QEMU and the server.
- ☐ **Lab 2.5.4:** MADT parsed by hand; root-bridge windows and extended configuration space cross-checked.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: what hardware saves on a trap and what software must, delegation, how x86 finds a kernel stack, and how an MSI becomes a handler call on x86 and Arm.

CHAPTER 2.6

Bare-metal firmware

TIME	3.5 weeks
YOU BUILD	Firmware for the MSP432E401Y from a blank directory, with no vendor library: startup code, linker script, clocks, an interrupt-driven UART, I²C and SPI masters reading TI sensors and driving your FPGA, DMA-fed ADC streaming, a fault handler that survives reset, a watchdog, and a signed A/B bootloader with anti-rollback. Then a port to a Cortex-M0+ to see what a different core removes.
NEEDS	Two MSP-EXP432E401Y LaunchPads, the LP-MSPM0G3507, OpenOCD built from git master, `arm-none-eabi-gcc`, the TMP117 and INA228 breakouts, the EMC2101 fan controller with a 4-pin fan, your Chapter 2.2 FPGA peripheral, logic analyser.

WHAT THIS BUYS OMH

A baseboard management controller is a microcontroller-class SoC running this kind of code: power sequencing, fan control from temperature, sensor polling over I²C and PMBus, a serial console, a watchdog and a signed firmware update that must never brick the board. The drone's flight controller is the same code with a control loop in the middle. Module Zero's management board in Chapter 3.4 and the drone in Chapter 3.5 both run firmware you write here. This chapter builds three habits that shippable firmware depends on: reading the reference manual before writing code, owning the linker script, and treating every interrupt as a concurrency problem.

In this chapter you use no TI driverlib, SysConfig, SDK or Arduino code, only the registers and the reference manual. Afterwards, use whatever you like, because you will know what it hides.

SKIP TEST

1. From power-on, list what a Cortex-M4 does before the first instruction of `main`, and which steps are hardware and which are your startup code.
2. Write a linker script that places `.text` in flash and `.data` and `.bss` in SRAM and gives the startup code the symbols it needs. What is `KEEP` for?
3. A UART ISR and `main` share a ring buffer. Which variables need `volatile`, what does `volatile` not give you, and how do you make the head and tail updates safe without disabling interrupts?
4. Decode a hard fault: which registers give the cause and the faulting address, and how do you find the PC of the faulting instruction from the stacked frame?
5. Design an A/B firmware update with rollback protection that cannot be bricked by power loss at any instant. What state must be atomic, and how do you make it so?

If all five are easy, do Labs 2.6.1, 2.6.5 and 2.6.6.

Core ideas

Anatomy of a microcontroller. The MSP432E401Y has a Cortex-M4F core with a Nested Vectored Interrupt Controller (NVIC), a SysTick timer, an MPU and an FPU; internal flash, SRAM and EEPROM; and peripherals on AHB and APB buses, all at fixed addresses in one 4 GiB map. The MSP432E4 Technical Reference Manual (TI SLAU723A, 1,817 pages) documents everything outside the core; Arm's Cortex-M4 Devices Generic User Guide documents the core. The memory-map section is the first thing you read for any new part.

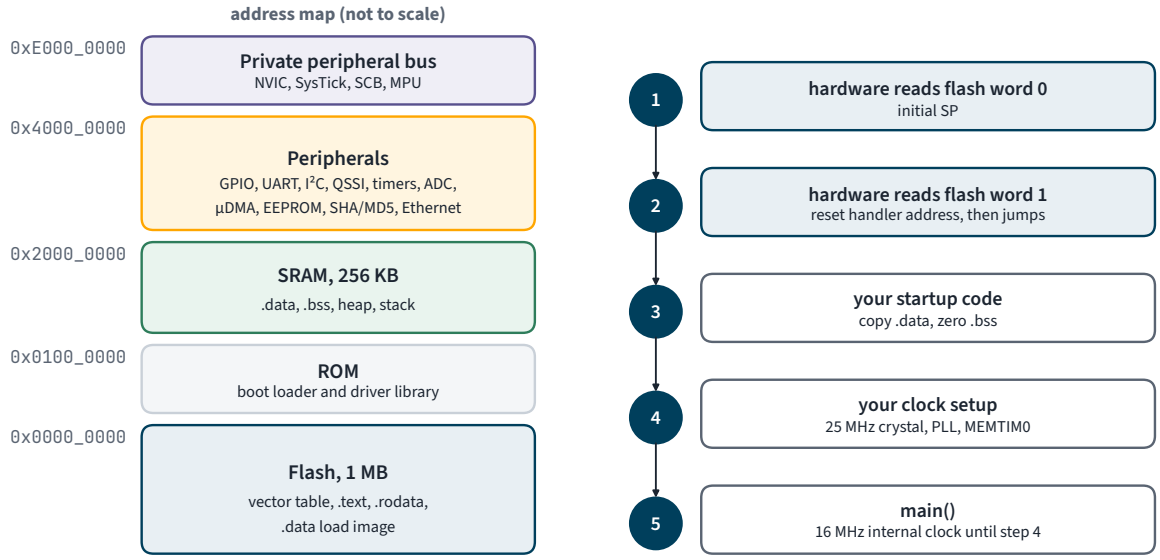


Figure 2.6.1. The MSP432E401Y's address map and the reset sequence. Hardware loads the stack pointer and reset vector from the first two words of flash; everything after that is your startup code.

Reset. The core reads the vector table at address 0: word 0 is the initial stack pointer and word 1 the reset handler. Hardware loads SP and jumps. From there your code must copy `.data` from flash to SRAM, zero `.bss`, optionally run constructors, set up clocks and call `main`. At that point nothing else has been set up, and the chip is running from its internal 16 MHz oscillator with every peripheral unclocked.

Linker scripts. `MEMORY` names the regions with their addresses and sizes. `SECTIONS` places input sections into output sections in those regions, defines the symbols startup code uses (`_sdata`, `_edata`, `_sbss`, `_estack`) and controls alignment. `.data` has a load address in flash (`AT>`) and a run address in SRAM. `KEEP` stops `--gc-sections` discarding the vector table. The map file tells you where every byte went.

Clocks. To run at 120 MHz you start the main oscillator from the LaunchPad's 25 MHz crystal, configure the PLL and wait for lock, set the flash and EEPROM timing (MEMTIM0) for the new speed before switching, and then select the PLL as the system clock (RSCLKCFG). Each peripheral is clocked separately through its RCGC register, and you must wait for its PR (peripheral ready) bit. Any access to a peripheral with no clock raises a bus fault, which escalates to a hard fault unless bus faults are enabled. This is the most common first bug.

GPIO and alternate functions. Each pin has a direction, a digital-enable bit (forget it and the pin does nothing), pull-ups and pull-downs, drive strength, and an alternate-function select with a port-control field that muxes it to a peripheral. The data register is address-masked: address bits 9 to 2 select which pins a read or write touches, so you can change one pin without a read-modify-write. The datasheet's pin table, rather than the reference manual, says which function is available on which pin, and the LaunchPad user's guide (SLAU748B) says which pins reach the BoosterPack headers.

Peripherals as registers. A UART is a baud-rate divisor, a line control register, a status register and a data register. SPI (the QSSI module) adds clock phase and polarity. I²C adds open-drain signalling (Chapter 1.1), addresses, ACKs, clock stretching and a state machine that hangs easily if you do not handle every error. Timers have a prescaler, a counter, a load value and capture/compare, and PWM is a compare toggling a pin. The ADC's sample sequencers convert a programmed list of channels on a trigger. The micro-DMA (µDMA) controller copies between peripherals and memory without the core, including ping-pong buffers for continuous streams. Two watchdogs, one on the system clock and one on the independent precision oscillator, reset the chip if not fed.

Interrupts. The NVIC has per-interrupt enable, priority and pending bits, and higher priority preempts lower. The core stacks eight registers automatically and calls a plain C function. Latency is about 12 cycles plus whatever time you spend with interrupts disabled. ISRs must be short, communicate through ring buffers or flags, and never block. `volatile` forces the compiler to reload but gives no atomicity. For atomic

updates, use `LDREX` / `STREX`, disable interrupts for the shortest critical section, or design single-producer single-consumer buffers where head and tail each have one writer.

Polling, interrupts and DMA. Polling occupies the core, interrupts cost time per event, and DMA costs nothing per byte but needs setup. Interrupting once per byte is fine for a UART at 115200 and too costly for an ADC at 1 MSa/s.

Faults. Bus, usage and memory-management faults escalate to hard fault unless enabled. CFSR, HFSR, BFAR and MMFAR say what happened, and the stacked frame says where. A good fault handler prints them, writes them to a no-init RAM region and resets, so the next boot can report the crash.

Debugging. SWD is a two-wire debug port. The LaunchPad's on-board XDS110 implements it, and OpenOCD drives the XDS110 and exposes a GDB server. You can halt, step, read any register or memory, set hardware breakpoints and watchpoints, and flash. The XDS110 also provides the back-channel UART on UART0 (pins PA0 and PA1), which appears on your workstation as a serial port.

Bootloaders. A bootloader is the first program in flash, and it decides whether to jump to an application (set VTOR, set SP, jump) or to accept a new image. An A/B scheme keeps two application slots and a small metadata record. An update writes the inactive slot, verifies a signature over it, and then switches the active slot atomically. Rollback protection stores a monotonic minimum version and refuses older images. Power loss at any instant must leave a bootable system, so the switch is the last write and must be interpretable even if it was interrupted. The MSP432E401Y gives you useful hardware: flash erased in 16 KB blocks and programmed a 32-bit word at a time (or 32 words through the write buffer), 6 KB of EEPROM for small records that change, and a SHA-256 accelerator.

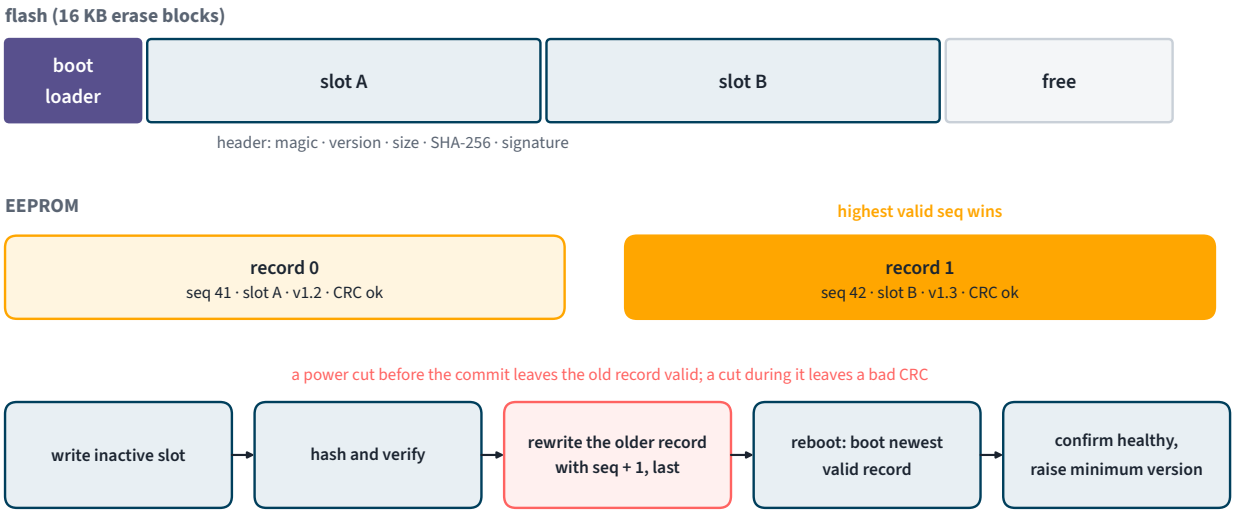


Figure 2.6.2. The A/B layout used in Lab 2.6.6 and the metadata commit. Two EEPROM records with sequence numbers and CRCs mean an interrupted commit always leaves one valid record, and the bootloader always knows which slot to try.

C and C++ in firmware. Build with `-Os` and `-ffunction-sections -fdata-sections -Wl,--gc-sections`, avoid `printf` with floats unless you need it, and add `-fno-exceptions -fno-rtti` for C++. C++ is fine in firmware with those flags and no unexpected heap use, and many teams use it. Static analysis (cppcheck, clang-tidy) and MISRA-style rules exist because firmware bugs are expensive to fix in the field.

A second core. The Cortex-M0+ in TI's MSPM0 is a smaller design: no FPU, no `LDREX` / `STREX`, no configurable fault status registers (every fault is a hard fault) and fewer instructions. Porting your startup code to it shows which of your assumptions were about the core and which about the chip.

An RTOS, afterwards. Once you have done it bare, FreeRTOS or Zephyr give you tasks, queues and timers. Zephyr's device tree and driver model mirror Linux's, which is what Chapter 4.3 teaches.

Reading

- TI SLAU723A, *MSP432E4 SimpleLink Microcontrollers Technical Reference Manual*: the chapters on System Control, Internal Memory, Processor Support and Exception Module, µDMA, ADC, GPIO, General-Purpose Timers, I²C, QSSI, UART, Watchdog Timers and SHA/MD5. Read each one fully on the day you use it.
- TI SLAU748B, the MSP-EXP432E401Y LaunchPad user's guide: pin assignments, BoosterPack headers and the back-channel UART jumpers. The MSP432E401Y datasheet's pin tables.
- Arm, *Cortex-M4 Devices Generic User Guide*: chapter 2 (core and exception model) and chapter 4 (NVIC, SysTick, the SCB fault registers).
- White, *Making Embedded Systems*, 2nd ed., chapters 2 to 8 and 10.
- Yiu, *The Definitive Guide to Arm Cortex-M3 and Cortex-M4 Processors*, chapters 4, 7, 8, 12, 13 and 22.
- Memfault's Interrupt blog: "From Zero to main()", "Cortex-M Fault Debugging" and the "Device Firmware Update Cookbook".

Labs

LAB 2.6.1 Blink from a blank directory — 5 h

Kit. MSP-EXP432E401Y, USB cable.

1. Create `startup.s` (vector table with SP and reset handler; copy `.data`, zero `.bss`, call `main`), `linker.ld` and `main.c`. Use no TI headers, and define the SYSCTL and GPIO register addresses yourself from the reference manual.
2. Enable the clock to GPIO port N, wait for its peripheral-ready bit, set PN1 as a digital output and toggle LED D1 with a busy loop. Use the address-masked data register to touch only PN1.
3. Build with `arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16 -nostdlib -ffreestanding`. Flash and debug with OpenOCD (`-f board/ti/msp432-launchpad.cfg`) and GDB. Break at the reset handler and `stepi` until you can narrate every instruction before `main`.
4. Read the map file and find the vector table and the load address of `.data`.

DONE WHEN

- [ ] D1 blinks, and the notebook narrates every instruction from reset to `main`.

EVIDENCE FOR THE NOTEBOOK

- The three source files; an annotated GDB session; the relevant map-file lines.

LAB 2.6.2 Clocks, a proper UART and time — 6 h

1. Start the 25 MHz main oscillator, configure the PLL for a 120 MHz system clock, set MEMTIM0 for that frequency, and switch. Output a divided system clock on a pin with the clock-output function and verify it on the oscilloscope.
2. Write a UART0 driver for the back-channel port: compute the integer and fractional divisors for 115200, interrupt-driven TX and RX with lock-free ring buffers, and overrun and framing error handling. Implement `_write` so `printf` from picolibc (or your own small formatter) prints through it.
3. Measure the actual baud rate with the logic analyser and compute the error.
4. Add SysTick at 1 kHz with a millisecond counter and `delay_ms`.

DONE WHEN

- [ ] The scope shows the expected clock output, and `printf` works at 115200 with a measured error under 1 %.

EVIDENCE FOR THE NOTEBOOK

- The divisor arithmetic; the clock capture; the baud measurement.

LAB 2.6.3 I²C and SPI masters: sensors, a fan and the FPGA — 8 h

Kit. TMP117, INA228, EMC2101 breakout with a 4-pin PWM fan, the ULX3S with Lab 2.2.5's bitstream, logic analyser.

1. Write an I²C master driver for I2C0 on PB2 (SCL) and PB3 (SDA), the BoosterPack 1 header. Handle NACK, bus busy, arbitration loss and timeouts, and implement the bus-clear procedure from Chapter 1.3.
2. Port your Chapter 1.3 TMP117 and INA228 drivers from Python to C on this bus. Read the EMC2101's fan tachometer and set its PWM duty.
3. Build a fan curve: the TMP117's temperature sets the fan duty, the INA228 reports the fan's power, and the loop logs all three each second. A BMC runs the same thermal loop at a larger scale.
4. Write an SPI master for SSI2 (PD3 clock, PD1 out, PD0 in) and talk to your FPGA peripheral: read the ID register, set GPIO, read the timer. Wire the FPGA's interrupt to a GPIO input and handle it.
5. Capture and annotate an I²C transaction with a NACK and an SPI transaction.

DONE WHEN

- [ ] Temperature drives fan speed with the power logged; the FPGA answers over SPI and its interrupt arrives on the GPIO; both captures are annotated.

EVIDENCE FOR THE NOTEBOOK

- Driver sources; a log of the fan curve; the two annotated captures.

LAB 2.6.4 DMA — 5 h

1. Configure an ADC sample sequencer, triggered by a timer, to sample a pin at 100 kSa/s into a 1,024-sample ping-pong µDMA buffer. Compute a running mean per half buffer and stream it out of the UART, also by DMA.
2. Measure CPU load: toggle a GPIO in each ISR and integrate on the scope, or count idle-loop iterations. Compare with per-sample interrupts at the same rate and note the rate at which they can no longer keep up.

DONE WHEN

- [ ] The stream runs continuously with no overruns, and the notebook has a CPU-load comparison.

EVIDENCE FOR THE NOTEBOOK

- The µDMA control-table setup annotated; the load measurements.

LAB 2.6.5 Faults and the watchdog — 5 h

1. Enable bus, usage and memory-management faults. Write one fault handler that prints CFSR, HFSR, BFAR, MMFAR and the stacked PC, LR and xPSR, writes them to a no-init RAM section and resets.

2. Trigger each of these: an unaligned access to a device register, a read from an unmapped address, execution from a region the MPU marks never-execute, a divide by zero with the trap enabled, and a stack overflow into an MPU guard region.
3. On boot, check the no-init region and report the previous crash.
4. Enable the PIOSC-clocked watchdog with a 500 ms timeout, fed from the main loop. Hang the loop on purpose and log the reset reason from the reset-cause register.

DONE WHEN

- [ ] Each fault is decoded correctly and survives the reset into the next boot's log; the watchdog reset is identified as such.

EVIDENCE FOR THE NOTEBOOK

- The fault handler; a log of each fault and the watchdog reset.

LAB 2.6.6 A signed A/B bootloader

2 weekends

1. Partition flash in 16 KB blocks: bootloader (32 KB), slot A and slot B. Keep two metadata records in EEPROM, each holding a sequence number, the active slot, the image's version and a CRC. Define an image header: magic, version, size, SHA-256, signature and entry vector.
2. Bootloader: pick the valid metadata record with the highest sequence number; hash the active image with the SHA-256 accelerator; verify an ECDSA P-256 signature (micro-ecc) against a public key compiled into the bootloader; check the version against the minimum stored in EEPROM; set VTOR and SP; jump. On failure, try the other slot, and if both fail, stay in update mode.
3. Update mode over UART: a framed protocol with a CRC that writes the inactive slot and verifies it, then commits by writing the older metadata record with a higher sequence number, last.
4. Anti-rollback: raise the minimum version only after the new image has booted and confirmed itself healthy.
5. Test power loss during the slot write, during the metadata write and just after. Every time, the board must boot something valid. Load an image signed with the wrong key and an older version; both must be refused.

DONE WHEN

- [ ] You can update from A to B and back over UART; every power-loss test boots; both rejection cases are demonstrated.

EVIDENCE FOR THE NOTEBOOK

- The layout and record format; the power-loss test log; the host-side update tool.

LAB 2.6.7 A second core: the Cortex-M0+

5 h

Kit. LP-MSPM0G3507.

1. Port Labs 2.6.1 and 2.6.2 to the MSPM0G3507 with no SDK, using its technical reference manual. Note what differs in the clock system, the boot configuration held in the NONMAIN flash region, and the core.
2. Show which of your Lab 2.6.5 fault decodes no longer exist on the M0+, and rewrite your ring buffer's atomic update without `LDREX` / `STREX`.

SAFETY

Do not erase or program the MSPM0's NONMAIN flash region. It holds the boot configuration, and wrong values can lock the device permanently. Your linker script and OpenOCD commands must touch MAIN flash only.

DONE WHEN

- [ ] The M0+ blinks and prints from your own startup code, and the notebook has a table of what the M4F gave you that the M0+ does not.

EVIDENCE FOR THE NOTEBOOK

- The port; the comparison table.

Problem set

1. Compute the UART's integer and fractional divisors for 115200 at 120 MHz with 16× oversampling, and the resulting error. Repeat for 921600. At what rate does the error exceed 2 %?
2. Design I²C pull-ups for a 3.3 V bus with 180 pF and a 400 kHz target, and check the sink-current limit.
3. Give a concrete interleaving that `volatile` on a shared counter does not prevent. Fix it with `LDREX` / `STREX`, and separately with a critical section. When is each appropriate, and what do you do on the M0+?
4. An ISR at priority 2 takes 5 µs; one at priority 1 (higher) arrives 1 µs in. Draw the timeline. Swap the priorities; make them equal. What is the worst-case latency for the lower one if `main` also disables interrupts for 20 µs?
5. A stacked frame shows PC = 0x0000_1234, LR = 0xFFFF_FFF9, CFSR = 0x0000_8200 and BFAR = 0x4002_0000. What happened, and which stack was in use?
6. A UART receives at 921600 and the main loop services its ring buffer every 2 ms. Size the buffer. What if the loop can stall for 50 ms during a flash erase?
7. Design the metadata state machine for Lab 2.6.6 so that power loss at any point leaves a state the bootloader can interpret. Explain why a "confirmed good after first boot" flag handles firmware that boots but crashes.
8. A 16 KB flash erase takes milliseconds and stalls flash reads on the same bank. What does that mean for interrupts and the watchdog during an update, and how do you design around it?
9. Why is EEPROM a better home than flash for the metadata records and the anti-rollback counter? What limits how often you may write it?

Deliverables and stretch

Deliverables. A library-free firmware project with linker script, startup, clocks, UART, I²C, SPI, DMA, fault handler and watchdog, driving TI sensors, a fan and the FPGA; the signed A/B bootloader with its power-loss log and host tool; annotated captures; the M0+ port and comparison table.

Stretch. Serve the fan-curve data over Ethernet with lwIP on the MSP432E401Y's integrated MAC and PHY, as a first step toward Chapter 4.7's management interface. Implement PMBus (SMBus with packet error checking) against a PMBus-capable regulator. Write a host-side hardware abstraction so the sensor and fan logic compiles and unit-tests on the workstation with mocked registers.

COMPLETION CHECKLIST

- [ ] **Lab 2.6.1:** blink from your own startup and linker script; every pre-`main` instruction narrated.
- [ ] **Lab 2.6.2:** 120 MHz verified on the scope; `printf` over the back-channel UART with under 1 % baud error.
- [ ] **Lab 2.6.3:** fan curve driven by the TMP117 with INA228 power logged; FPGA over SPI with its interrupt handled.
- [ ] **Lab 2.6.4:** continuous DMA stream with a CPU-load comparison.
- [ ] **Lab 2.6.5:** five faults decoded across reset; watchdog reset identified.
- [ ] **Lab 2.6.6:** signed A/B update in both directions; all power-loss tests boot; wrong key and old version refused.

- [ ] **Lab 2.6.7:** M0+ port and the comparison table.
- [ ] **Problem set:** all nine answered.
- [ ] **You can explain**, without notes: everything between reset and `main`, why `volatile` is not atomicity, how to decode a fault, and why your bootloader cannot be bricked by a power cut.

CHAPTER 2.7

Boot and platform firmware

TIME	3.5 weeks
YOU BUILD	A boot sector and a two-stage loader into long mode; a UEFI loader; Secure Boot with your own keys on QEMU and the server; measured boot with an event-log replay and a remote-attestation verifier that admits and refuses machines; an Arm boot chain with U-Boot, a device-tree overlay and signed FIT images; a zero-touch netboot that installs, attests and joins; a Redfish client and an OpenBMC build with your own sensor.
NEEDS	The lab server with its TPM and BMC, QEMU with OVMF and swtpm, the Raspberry Pi 5, the managed switch, a TMP117 breakout.

WHAT THIS BUYS OMH

OMH's site promises that a module "powers on, attests its firmware to the control plane, finds its peers across the backplane and starts serving", with OpenBMC, Redfish and measured boot on the specification sheet. This chapter builds each part of that paragraph. By its end you will have netbooted a machine with no local configuration, measured its firmware and kernel into a TPM, verified a quote against a policy on another machine, refused an unsigned bootloader, driven a BMC over Redfish and built OpenBMC. Module Zero's join protocol in Chapter 4.7 is assembled from these labs.

SKIP TEST

1. Why can x86 firmware not use RAM for its first few thousand instructions, and what does it use instead?
2. Explain the difference between Secure Boot and measured boot, and give a failure each catches that the other cannot.
3. PCR 4 holds a value after boot. Describe exactly how it was computed and what you need to reproduce it from the event log.
4. A remote verifier receives a TPM quote. List what it must check before believing the PCR values, including how it knows the quote came from a real TPM.
5. Draw the packet sequence for a PXE boot from power-on to the kernel's first instruction, naming the protocols and DHCP options.

If all five are easy, do Labs 2.7.4, 2.7.6 and 2.7.7.

Core ideas

Power-on on x86. The first fetch is at 0xFFFF_FFF0 in 16-bit real mode, from SPI flash mapped below 4 GiB. DRAM is not usable until the memory controller is trained. Firmware runs early code with the cache configured as RAM (no-evict mode), initialises memory through a vendor blob (Intel FSP or AMD's equivalent) and relocates into DRAM. It then sets up the platform (SMM, microcode, ACPI tables, PCIe enumeration and BAR assignment), and only after that searches for something to boot.

UEFI. UEFI is a specification for firmware structure and for the interface it offers bootloaders and operating systems. Its phases are SEC (security and cache-as-RAM), PEI (memory initialisation), DXE (driver execution, most of the work), BDS (boot device selection) and then the OS loader. Boot services (memory allocation, protocols, file access, `ExitBootServices`) exist until the OS takes over, and runtime services (variables, time, capsule update, reset) persist. An EFI application is a PE/COFF binary on a FAT EFI System

Partition, and the boot order is stored in NVRAM variables. EDK2 is the reference implementation and OVMF its QEMU build.

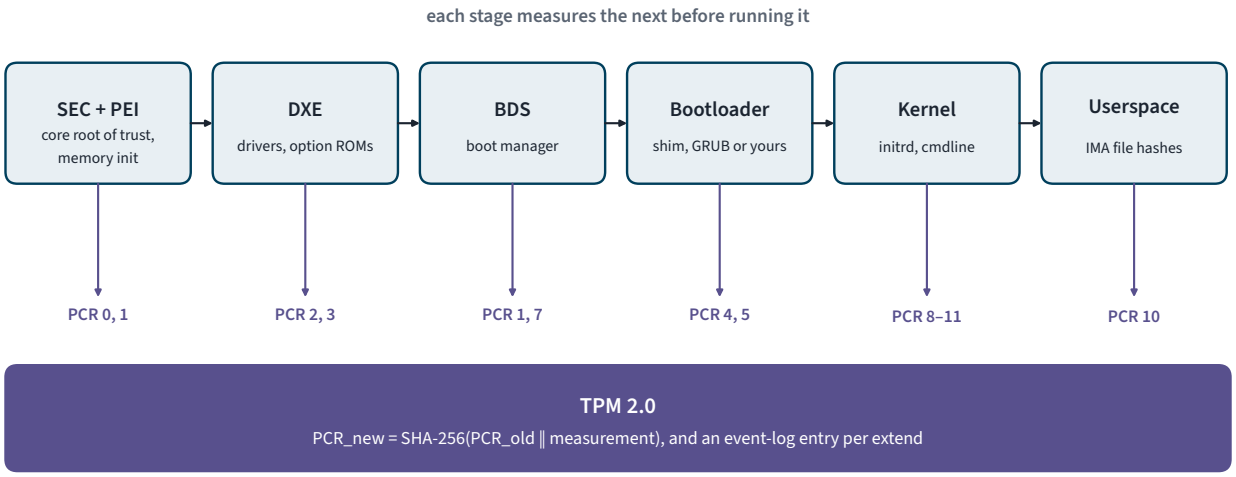


Figure 2.7.1. The x86 boot phases and what each extends into the TPM. Every stage measures the next before running it, so the final PCR values commit to the exact sequence of code that ran.

coreboot and LinuxBoot. coreboot does the minimum initialisation and hands off to a payload: a UEFI implementation, GRUB, or a Linux kernel acting as the bootloader (LinuxBoot). Hyperscalers use it because auditable firmware is the base of a measured chain, and an OMH platform team should evaluate it.

Secure Boot. Secure Boot is a signature chain enforced by firmware. The Platform Key (PK) owns the Key Exchange Keys (KEK), which own the allowed database (db) and the forbidden database (dbx). Firmware runs only PE binaries signed by a db key and not listed in dbx. Distributions boot through Microsoft-signed shim, which verifies GRUB and the kernel against its own keys and the Machine Owner Key list. On your own hardware you enrol your own PK and sign everything. Secure Boot checks whether each binary was allowed to run, and it records nothing about what actually ran.

The TPM and measured boot. A TPM is a small, slow, tamper-resistant chip whose keys never leave it and whose Platform Configuration Registers can only be extended: $\mathrm{PCR}_{\mathrm{new}} = \mathrm{SHA256}(\mathrm{PCR}_{\mathrm{old}} \,\|\, \mathrm{measurement})$. Firmware extends itself into PCR 0, its configuration into PCR 1, option ROMs into 2 and 3, the bootloader into 4, the partition table into 5, and Secure Boot state into 7. The bootloader or a unified kernel image extends the kernel, initrd and command line (PCRs 8, 9 or 11), and IMA extends file hashes into PCR 10. Each extend is logged in the TCG event log. Because extend is a hash chain, the final values commit to the exact sequence, and a verifier replaying the log must arrive at the same values.

Attestation. The Endorsement Key (EK) is unique per TPM, with a manufacturer certificate proving the TPM is genuine. An Attestation Key (AK) is created in the endorsement hierarchy and proven to belong to the same TPM by credential activation. A quote is a TPM-signed statement of selected PCRs plus a verifier-supplied nonce. Sealing binds a secret to PCR values so that it unseals only in the same boot state, which is how a disk key can be released only to your kernel. RFC 9334 names the parties: Attester, Verifier and Relying Party, exchanging evidence, endorsements, reference values and appraisal policy. OMH's control plane is the Verifier, the module the Attester and the quorum the Relying Party.

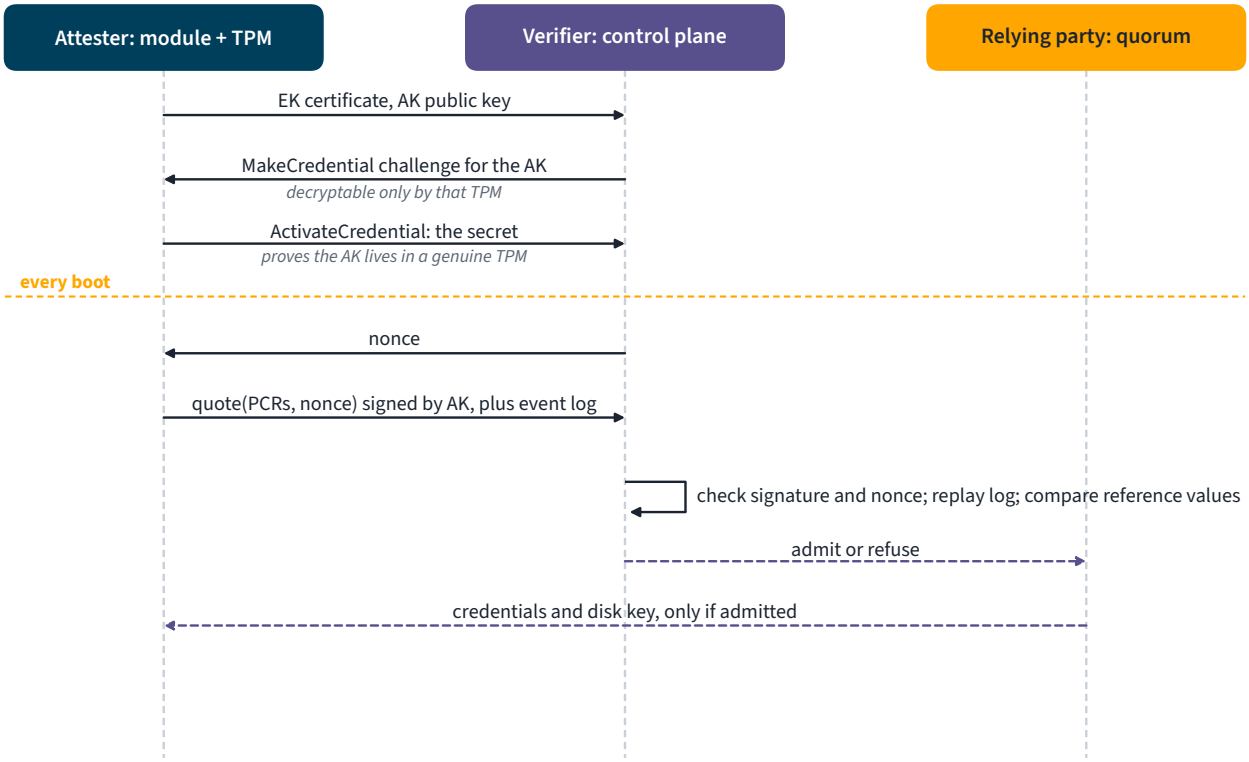


Figure 2.7.2. The attestation protocol Lab 2.7.4 builds. The nonce defeats replay; credential activation ties the AK to a genuine TPM; the event-log replay ties the PCRs to known software.

Measured boot in the small. Microcontrollers and BMCs do the same with DICE (each stage derives a key from the previous stage's secret and the next stage's hash) and with hardware roots of trust such as OCP's Caliptra. SPDM lets a host attest its PCIe devices. Your Chapter 2.6 bootloader is the first link of such a chain.

Arm and RISC-V boot chains. On Arm, a boot ROM loads a first stage, which loads Trusted Firmware-A (BL2, then BL31 as the EL3 monitor, optionally a secure OS) and then U-Boot (BL33) at EL2, which loads a kernel and a device tree. The Pi differs in that its GPU firmware does the early work. RISC-V runs a zero-stage loader, OpenSBI in M-mode providing the Supervisor Binary Interface, then U-Boot in S-mode. U-Boot is a small OS in its own right, with drivers, filesystems, networking, scripting, its own device tree, and verified boot of signed FIT images.

What the kernel expects. On x86 the kernel expects the boot protocol in `Documentation/arch/x86/boot.rst` (a real-mode header, `boot_params`, the command line, an initrd and an entry point), or it uses the EFI stub to run as an EFI application. On arm64 it expects an image header and the DTB address in `x0`. On RISC-V it expects the hart ID in `a0` and the DTB in `a1`, with OpenSBI resident.

Netboot and zero-touch. The NIC's option ROM or the UEFI network stack does DHCP. The server replies with an address and a boot file (options 66 and 67, chosen by the client architecture in option 93), and the client fetches it over TFTP (PXE) or HTTP (UEFI HTTP boot). iPXE, chain-loaded, scripts the rest, fetching a kernel and initrd over HTTP with per-machine URLs. A node with no local state then fetches its configuration from the control plane, which is the zero-touch join, and the control plane can require attestation before handing out anything sensitive.

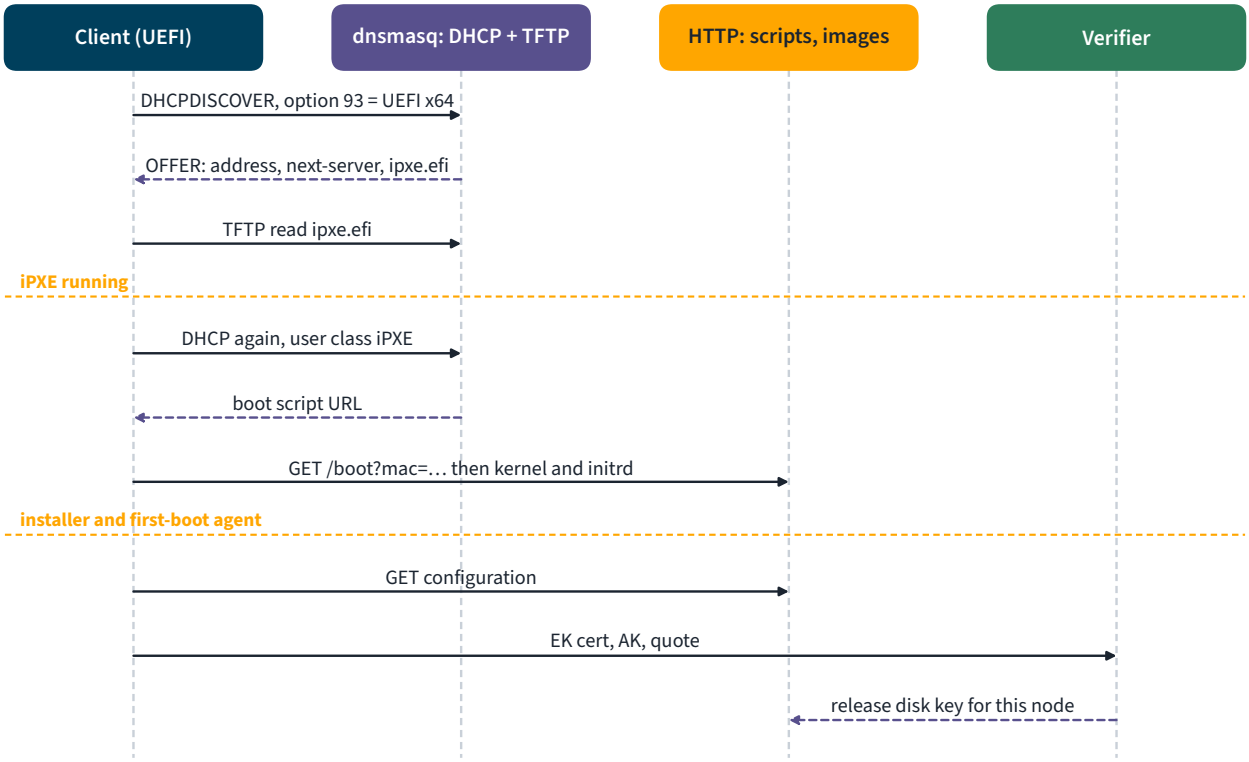


Figure 2.7.3. UEFI netboot with iPXE chain-loading, from DHCP to a running installer. Lab 2.7.6 builds every server in this picture.

The BMC. The BMC is a separate SoC (an ASPEED AST2500 or AST2600 on nearly every server) with its own Ethernet, flash and Linux, powered whenever the chassis has standby power. It controls host power and reset, reads sensors over I^2C and PMBus, drives fans, and provides a serial console over the network, virtual KVM and media, and firmware update for itself and the host. Host and BMC talk over LPC or eSPI, I^2C, and MCTP with PLDM. IPMI is the legacy protocol, and Redfish is its RESTful, schema-driven replacement. OpenBMC is the Linux Foundation's BMC distribution: phosphor daemons on D-Bus, `bmcweb` serving Redfish, entity-manager and hwmon for sensors, and a firmware-update flow. OCP's DC-SCM specification puts the BMC on a replaceable card, and a rack-module company should read it before designing anything.

Firmware storage and update. Firmware is stored in SPI NOR flash, often two chips for A/B, write-protected under BMC or strap control. UEFI capsules and Redfish `UpdateService` are the sanctioned paths. A rolling fleet update that drains and rejoins one module at a time is orchestration on top of these plus re-attestation after reboot.

Reading

- Zimmer, Rothman and Marisetty, *Beyond BIOS*, 3rd ed., chapters 1 to 4, 7 and 9.
- UEFI specification: chapter 2 (overview), boot and runtime services (skim), and the Secure Boot chapter.
- Arthur and Challener, *A Practical Guide to TPM 2.0* (free from Apress): chapters 1 to 3, 9 to 12 and 21.
- TCG PC Client Platform Firmware Profile, sections 2, 3 and 7; RFC 9334 in full.
- Linux `Documentation/arch/x86/boot.rst`, `arm64/booting.rst` and `riscv/boot.rst`; coreboot's getting-started and payloads pages; the LinuxBoot book.
- OpenBMC architecture, `bmcweb` and code-update documents; DMTF Redfish specification DSP0266 chapters 1 to 7 and the Redfish mockup server; OCP DC-SCM 2.0 (skim); iPXE scripting and chain-loading.

Labs

LAB 2.7.1 A boot sector, then a two-stage loader — 1 weekend

1. Write a 512-byte MBR boot sector that prints a string through BIOS interrupt 10h and then through the 16550 at 0x3F8 directly. Boot it in QEMU with `-drive format=raw` and explain every byte, including the signature at offset 510.
2. Stage two loads more sectors with BIOS 13h, collects the BIOS memory map with interrupt 15h function E820h and enables A20. It then sets up a GDT, enters protected mode, builds identity-mapped page tables and enters long mode. Finally it loads an ELF64 from a known LBA, copies its segments and jumps, passing the memory map to the payload. Your Lab 2.5.3 program is the payload; your Chapter 4.1 kernel will be.
3. Use QEMU's `-d int,cpu_reset` and the monitor's `info registers` when it triple-faults.

DONE WHEN

- [ ] Your Lab 2.5.3 program runs from your own boot sector in QEMU, and the notebook has an annotated hex dump of the sector.

LAB 2.7.2 A UEFI loader — 1 weekend

1. With gnu-efi (or EDK2), write an EFI application that prints the firmware vendor and revision, dumps the memory map with types, and finds the ACPI RSDP in the configuration table.
2. Load your ELF64 payload from the ESP through the Simple File System protocol, allocate pages and copy its segments.
3. Get the framebuffer through GOP and build a boot-information structure (memory map, framebuffer, RSDP, command line). Call `ExitBootServices`, handling the retry when the map changes, then set up your own page tables and jump to the payload.
4. Run it under OVMF in QEMU, then on the server from a USB stick. This loader boots your Chapter 4.1 kernel.

DONE WHEN

- [ ] The loader launches your payload on QEMU and on the server, and the payload prints the memory map it was handed.

LAB 2.7.3 Secure Boot with your own keys — 1 weekend

1. Generate PK, KEK and db key pairs with `openssl`, and produce EFI signature lists and authenticated variables with `efitools`.
2. In OVMF's Secure Boot build, enter setup mode and enrol your keys. Sign your Lab 2.7.2 loader with `sbsign` and boot it. Then boot the unsigned version and see it refused, and then sign it with a key not in db.
3. Repeat on the server. Back up the vendor keys first, write and test the recovery procedure, and then take ownership.
4. Boot a distribution kernel through shim and MOK on the server and record which key verified which binary at each step.

DONE WHEN

- [ ] Your loader boots only when signed by your db key, on OVMF and on the server, and your recovery procedure has been tested.

LAB 2.7.4 Measured boot and remote attestation — 2 weekends

1. Run QEMU with `swtpm` as a TPM 2.0 and boot a distribution. Read PCRs with `tpm2_pcrread` and parse the event log from `/sys/kernel/security/tpm0/binary_bios_measurements` with `tpm2_eventlog`.
2. Write a Python script that replays the raw event log and recomputes PCRs 0 to 7. It must match `tpm2_pcrread`. Change the kernel command line, reboot, and show which PCR changed and which event explains it.
3. Seal a secret to PCRs 0, 4 and 7 and unseal it. Boot your Lab 2.7.2 loader instead and show that unsealing fails. On the server, seal a LUKS key with `systemd-cryptenroll --tpm2-pcrs=0+7`.
4. Build a verifier service that receives the EK certificate and AK public key and verifies the EK chain (a manufacturer CA, or your own for swtpm). It runs credential activation to prove the AK is in that TPM, sends a nonce, and receives a quote and event log. It then checks the signature, nonce and PCR digest, replays the log against stored reference values per firmware version, and returns admit or refuse.
5. Run it against QEMU and the server. Change the server's command line and show refusal, then update the reference values and show admission.

DONE WHEN

- [ ] The verifier admits a machine in a known state and refuses one that differs, and the notebook diagrams what each protocol message proves.

LAB 2.7.5 Arm boot chain and a device-tree overlay — 1 weekend

Kit. Raspberry Pi 5, TMP117 breakout.

1. Build Trusted Firmware-A, U-Boot and a mainline kernel for QEMU's `virt` machine and boot with an explicit device tree. Inspect it with `dtc -I dtb -O dts`.
2. Build U-Boot for the Pi and boot Linux through it. In U-Boot's shell inspect the environment, `fdt print`, read a peripheral register with `md`, and load a kernel with `dhcp` and `tftpboot`.
3. Write an overlay for a GPIO LED and the TMP117 on I^2C (`compatible = "ti,tmp117"`), apply it from U-Boot with `fdt apply`, and confirm the kernel's driver finds the sensor under `/sys/bus/iio`.
4. Make a FIT image of kernel, DTB and initrd signed with your key; enable verified boot in U-Boot and show it refuses an unsigned FIT.

DONE WHEN

- [ ] The Pi boots through U-Boot with your overlay and a signed FIT, the TMP117 appears in IIO, and an unsigned FIT is refused.

LAB 2.7.6 Zero-touch netboot — 2 weekends

Kit. Managed switch with a lab VLAN, the server, the Pi or workstation as provisioning server.

1. Run `dnsmasq` for DHCP and TFTP on the lab VLAN, serving `ipxe.efi` to UEFI clients (matched on option 93), with iPXE chain-loading a script from a small web service you write.
2. The service returns a per-machine script keyed by MAC address (later by attested identity) that fetches a kernel and initrd over HTTP with a command line naming a configuration URL. Boot the server this way with no local disk involved.
3. In that live environment, a first-boot agent fetches its configuration, partitions the disk, writes your OS image, installs your loader or systemd-boot, and enrols the TPM with your Lab 2.7.4 verifier. On reboot, the installed system attests and the verifier records "joined".
4. Make the join conditional, so that the configuration service releases the disk-encryption key only after a passing quote.

DONE WHEN

- [ ] A wiped server netboots, installs itself, attests, receives its secrets and appears as joined, with no keyboard involved.

LAB 2.7.7 BMC and Redfish, then OpenBMC — 2 weekends

1. Against the server's BMC, walk the Redfish tree (`Systems`, `Chassis`, `Managers`) with `curl`; read power state, thermal sensors and PSU readings; power-cycle the host through an Action; open a serial-over-LAN session; read the event log. Write a small client that graphs inlet temperature and fan speed over time.
2. Compare with `ipmitool -I lanplus sensor list` and `sol activate`, and note what Redfish models that IPMI does not.
3. Build OpenBMC for a QEMU machine (`romulus-bmc` or `ast2600-evb`), boot it, explore D-Bus with `busctl tree` and hit `bmcweb`'s Redfish endpoints. Trace how a sensor flows from hwmon to D-Bus to Redfish.
4. Write a small phosphor-style D-Bus daemon in C++ with sdbusplus that publishes a sensor, and see it in Redfish.
5. Trace how a host BIOS update would be delivered through Redfish `UpdateService`.

DONE WHEN

- [ ] Your client controls and monitors the server, OpenBMC boots in QEMU with your sensor visible in Redfish, and you can describe the update flow end to end.

Problem set

1. Explain cache-as-RAM: what state the cache must be in, why evictions must be prevented, and what breaks if memory training fails.
2. Show that swapping two measurements changes the final PCR, and that a verifier with the log can detect a missing event but one with only the final PCR cannot say which.
3. An attacker controls the OS but not the firmware. Which PCRs can they influence? Can they forge a quote for a clean state, or replay yesterday's? What in your protocol prevents each?
4. Compare a PCR 7 sealing policy with a PCR 4 one across a bootloader update and a Secure Boot key rotation. How would OMH's rolling update avoid re-provisioning secrets? Read about `PolicyAuthorize`.
5. Draw the DHCP, TFTP and HTTP exchanges in UEFI HTTP boot with iPXE chain-loading, including which server answers each and which fields identify the client.
6. What does Secure Boot protect against that measured boot does not, and the reverse? Why does a fleet need both and a verifier?
7. U-Boot has its own device tree and the kernel gets another. Why two, and what happens if they disagree about a UART's address?
8. A rolling update changes one module's firmware. Design reference-value management that admits both old and new during the rollout and refuses an unknown third state.
9. A BMC reaches the host's power, console and firmware over the network. List the ways it can be attacked and what OpenBMC and DC-SCM do about each.

Deliverables and stretch

Deliverables. Boot sector and two-stage loader; UEFI loader; signed and unsigned boot demonstrations; the event-log replay script; the verifier with its documented protocol and a recorded admit and refuse; the

netboot service and first-boot agent with the join log; the Redfish client and the OpenBMC build with your sensor.

Stretch. Build coreboot for QEMU with a LinuxBoot payload measured into the TPM. Add DICE-style key derivation to your Chapter 2.6 bootloader and sign a firmware attestation the host can verify. Read SPDM and the `spdm-emu` project and describe how a host would attest an NVMe drive's firmware.

COMPLETION CHECKLIST

- ☐ **Lab 2.7.1:** the payload runs from your boot sector; the sector annotated byte by byte.
- ☐ **Lab 2.7.2:** the UEFI loader launches the payload on QEMU and the server.
- ☐ **Lab 2.7.3:** signed-only boot with your keys on both, and a tested recovery procedure.
- ☐ **Lab 2.7.4:** event-log replay matching the TPM; sealing demonstrated; the verifier admitting and refusing.
- ☐ **Lab 2.7.5:** the Pi through U-Boot with your overlay; TMP117 in IIO; unsigned FIT refused.
- ☐ **Lab 2.7.6:** a wiped server netboots, installs, attests and joins, hands-off.
- ☐ **Lab 2.7.7:** Redfish client; OpenBMC with your D-Bus sensor.
- ☐ **Problem set:** all nine answered.
- ☐ **You can explain**, without notes: the UEFI phases, how a PCR is extended and replayed, what each attestation message proves, and what a BMC can do to a host.

UNIT 2

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and labs repository, and you can do each review task below without notes. Date each line as you finish it. If someone who builds hardware for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
2.1	Digital logic	☐	☐	
2.2	HDL and FPGA	☐	☐	
2.3	Build a CPU	☐	☐	
2.4	Memory and interconnect	☐	☐	
2.5	Privilege and the platform	☐	☐	
2.6	Bare-metal firmware	☐	☐	
2.7	Boot and platform firmware	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook.

- ☐ Explain metastability and show, with your formal proof, why your asynchronous FIFO's pointers cross clock domains safely.
- ☐ Walk a load-use hazard and a taken branch through your pipeline cycle by cycle, and say what your forwarding and flush logic does in each.
- ☐ Explain a cache miss on the server from the core to DRAM and back, using your Lab 2.4.3 measurements.
- ☐ From reset to `main` on the MSP432E401Y, and from power-on to a login on the server, name every stage and what each one verifies before it runs the next.
- ☐ Show how your A/B bootloader survives a power cut at every point, and how a TPM quote convinces a remote verifier of what booted.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____